

Генеративные модели

или тренды в RecSys

ШАД, Рекомендательные системы
2025/26, лекция 13

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

1. Генеративные модели
2. Генеративное ранжирование
3. Генеративный retrieval и semantic IDs
4. Одностадийные рексистемы и OneRec
5. Разговорные рексистемы

Что такое генеративные модели

Часть 1

Определение

Генеративная модель:

- Есть данные $\{x \in X\}$
- Есть распределение этих данных $p(x)$
- Генеративная модель позволяет сэмплировать из этого распределения, $x \sim p(x)$

Когда говорят “генеративная модель”, можно спросить “генеративная модель чего?”

LLM как генеративная модель

LLM – это генеративная модель над текстом:

- Есть корпус текстов из распределения $p(w)$
- Авторегрессивная генеративная модель:
 - $p(w_1, \dots, w_n) = p(w_1) \cdot p(w_2 | w_1) \cdot \dots = \prod_{i=1}^n p(w_i | w_1, \dots, w_{i-1})$
- Чтобы сэмплировать текст, нужно сгенерировать (сэмплировать) первое слово, потом второе при условии первого и так далее



Условная генерация

Обычно мы применяем LLM по-другому:

- Сэмплируем из $p(x | c)$, где c – некое условие
- В случае LLM это промпт – инструкция, задача, вопрос, условие
- Дискриминативные вероятностные модели тоже имеют форму вида $p(x | c)$, но предполагают что $x \in X$ – это объекты с простой структурой (таргеты, классы)

Компьютерное зрение

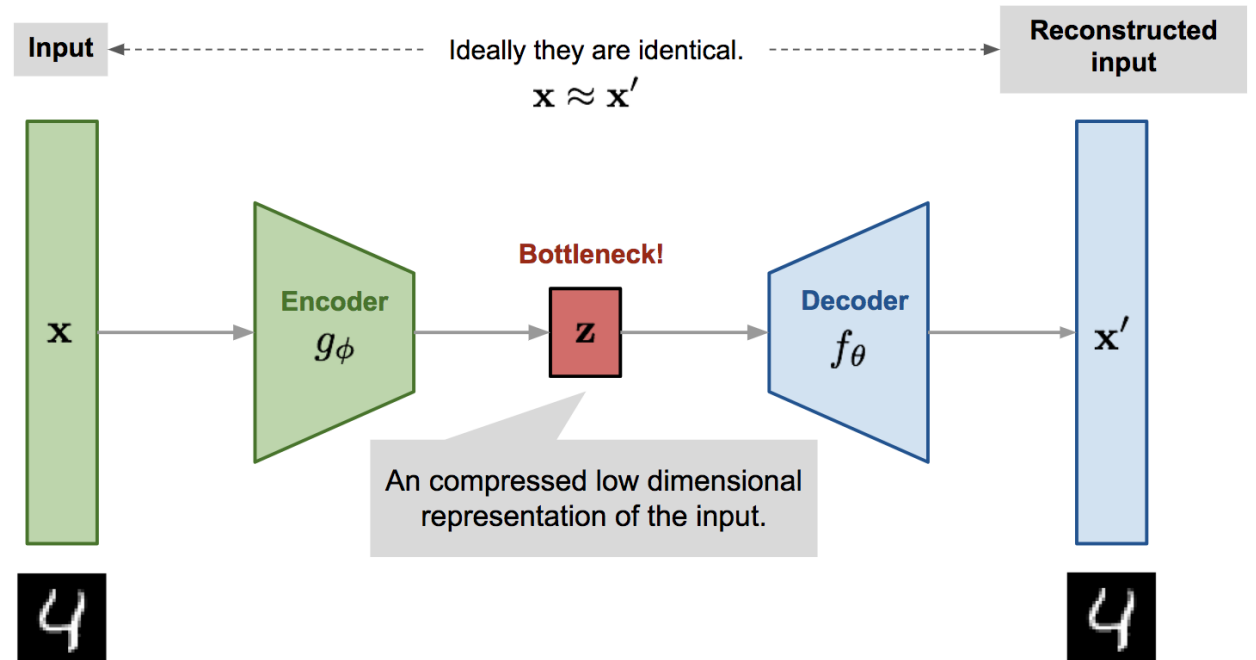
В компьютерном зрении уже долгое время развивают генеративные модели. Например:

- Вариационные автокодировщики (VAE)
- Генеративно-состязательные сети (GAN)
- Диффузионные модели

Базовая цель – генерировать реалистичные изображения.

Автокодировщики

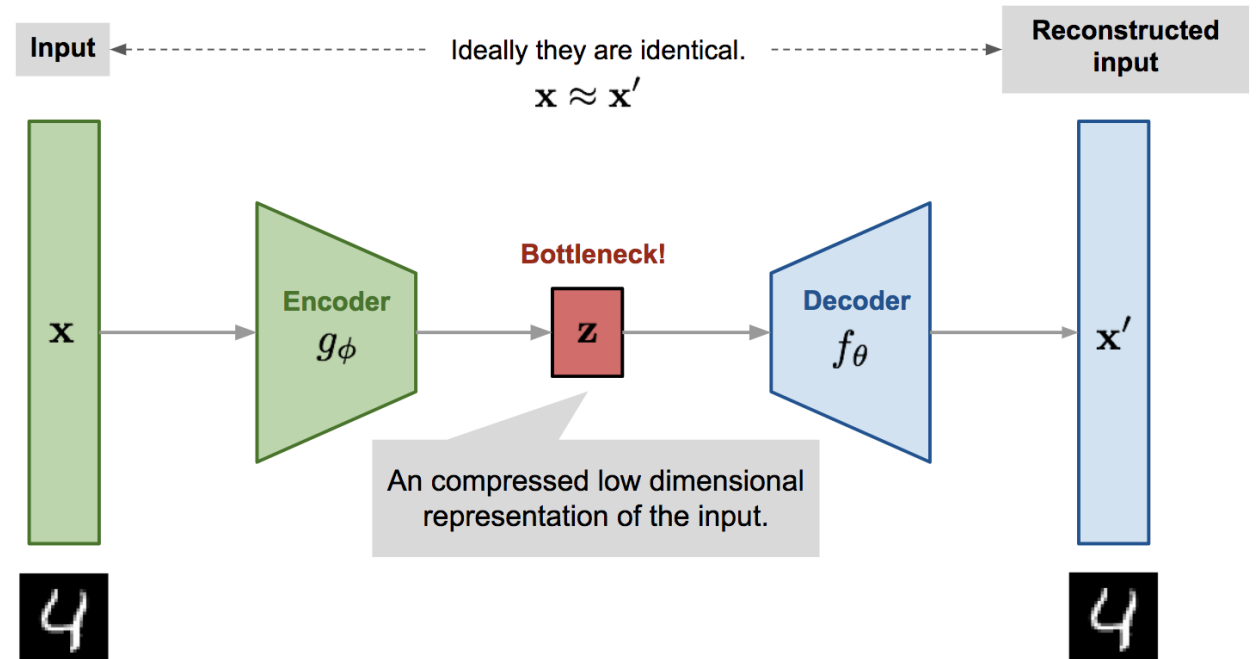
- Encoder $f: X \rightarrow Z$
- Decoder $h: Z \rightarrow H$
- Reconstruction loss:
 - $\|h(f(x)) - x\|_{l_2} \rightarrow \min$
- Это генеративная модель?



[From Autoencoder to Beta-VAE](#)

Автокодировщики

- Encoder $f: X \rightarrow Z$
- Decoder $h: Z \rightarrow H$
- Reconstruction loss:
 - $\|h(f(x)) - x\|_{l_2} \rightarrow \min$
- Это генеративная модель?
Нет. Не можем сэмплировать объекты из X



[From Autoencoder to Beta-VAE](#)

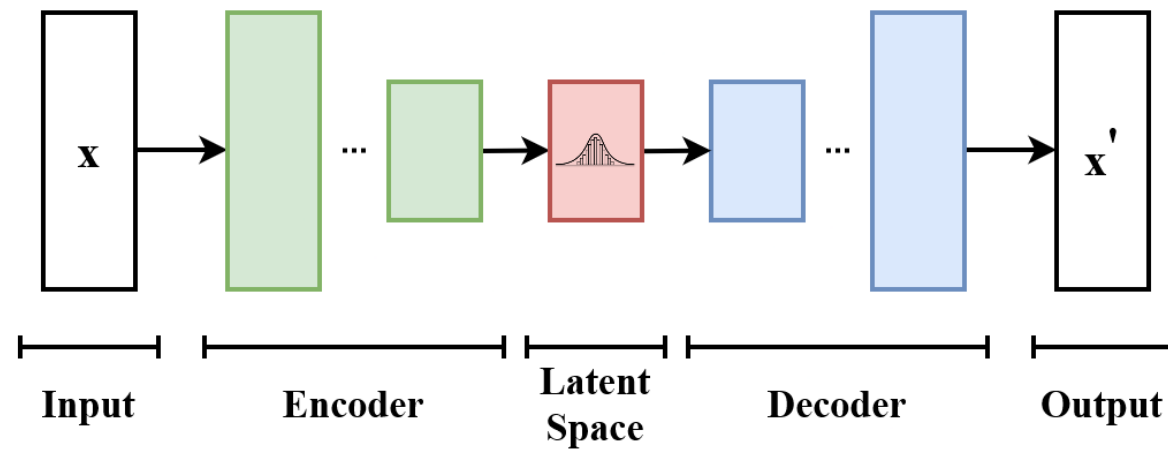
Вариационные автокодировщики

Генеративная модель:

$$p(x, z) = \underbrace{p(z)}_{\text{prior}} \underbrace{p_{\theta}(x | z)}_{\text{decoder}}$$

где $x \in \mathbb{R}^d$ – векторное представление объекта, $z \in \mathcal{Z}$ – скрытая переменная.

Если скрытая переменная дискретная, то это discrete VAE (dVAE).



https://en.wikipedia.org/wiki/Variational_autoencoder

DALLE-1

- Прорыв в генерации изображений
 1. Обучаем dVAE над изображениями
 - Энкодер – сверточная нейросеть, выдающая распределения для 1024 дискретных токенов (кодов)
 - Декодер – по кодам восстанавливает исходное изображение
 2. Учимся по текстовому описанию генерировать дискретный код изображения
 3. Применение: по описанию генерируем дискретный код, применяем dVAE декодер -> получаем картинку



a shiba inu wearing a beret and black turtleneck

Картинка из [DALL-E 2](#), говорим про [DALLE 1](#).

Генеративные рекомендации

Что из перечисленного является генеративной моделью?

- SASRec
- YoutubeDNN, ранжирующая нейросеть
- YoutubeDNN, двухбашенная нейросеть

[Self-Attentive Sequential Recommendation](#)

[Deep Neural Networks for YouTube Recommendations](#)

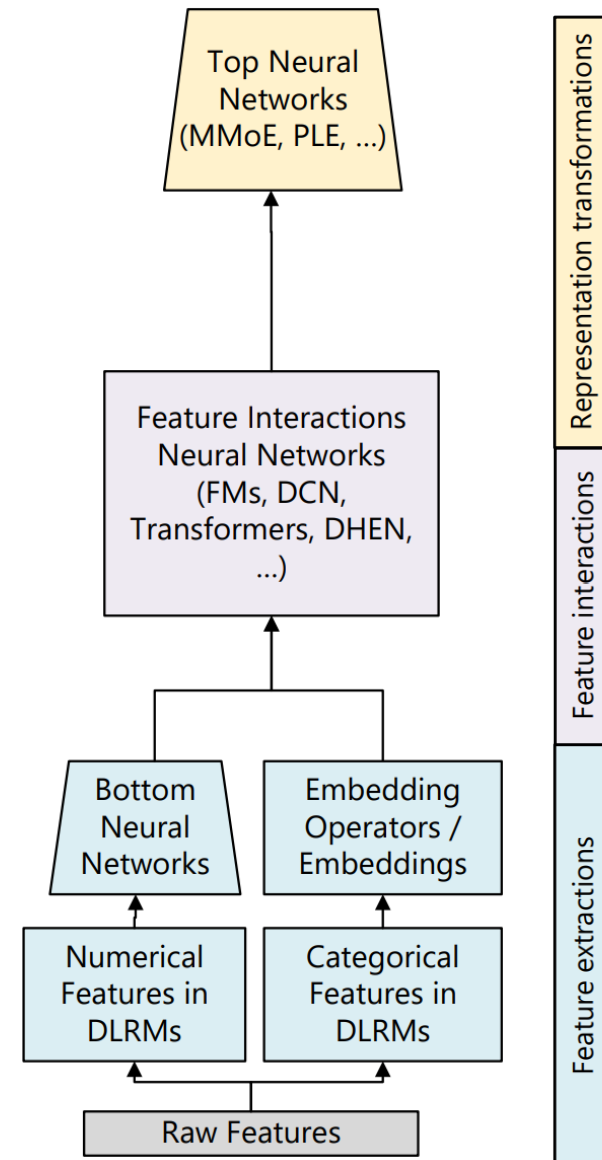
Генеративное ранжирование

Часть 2

Ранжирование

Ранжирующая модель:

- Принимает на вход user, item, user-item признаки
- Использует специализированные слои для моделирования взаимодействия признаков
- И выдает оценки разных сигналов
 - Возможно сразу агрегирует их в финальный скор

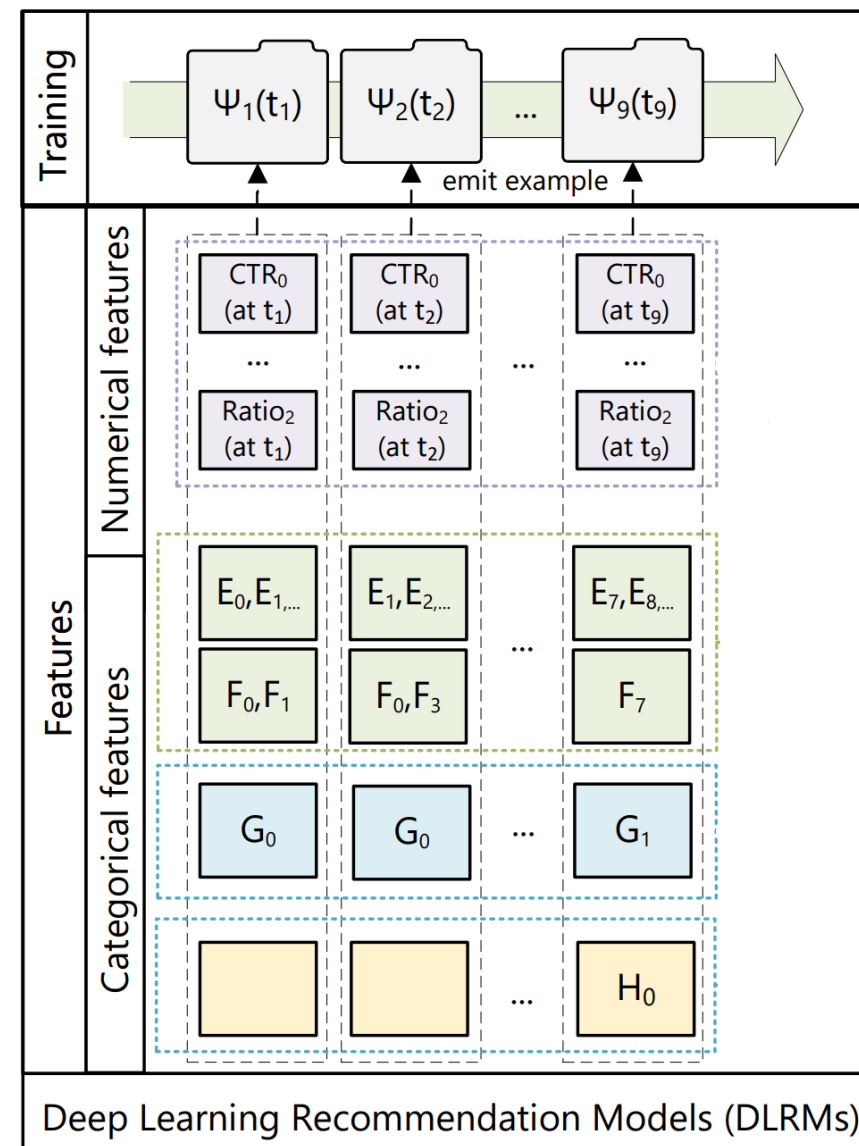


[Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations](#)

Ранжирование

Обучение – impression-level:

- Обучающий пример – залогированный показ рекомендации, для которого знаем фидбек (был ли лайк или нет, etc)
- Пользователю показали n айтемов – будет n обучающих примеров



Actions speak louder than words

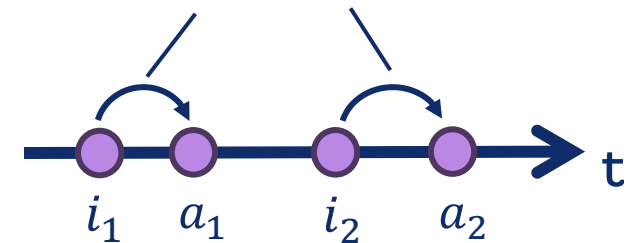
Обсудим сильно на шумевшую в свое время статью “Actions Speak Louder than Words...”:

- Подняли популярность концепта генеративных рекомендаций
- Замасштабировали модели
- Показали большие приросты метрик в проде

Item-action interleaving

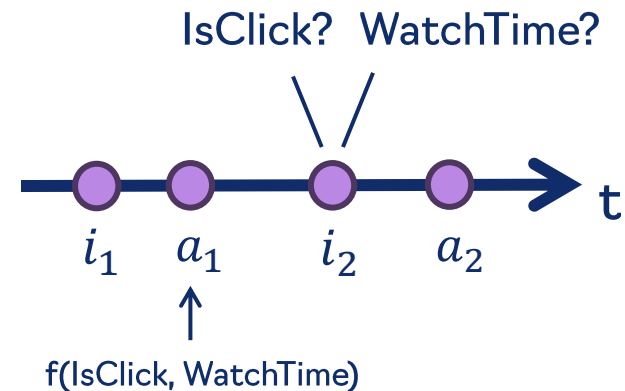
- Авторы представили пользователя в виде последовательности показов рекомендаций и фидбека
 - Фидбек пользователя как отдельный токен-событие
- Генеративная модель:
 - $P(user) = P(i_1) \cdot P(a_1 | i_1) \cdot P(i_2 | i_1, a_1) \cdot \dots$
 - НО: они не делают такую модель, а учат только $P(action | \dots)$
- $P(action | history, item)$ – это буквально **target-aware** постановка из Deep Interest Network
 - Теперь можно учить ее не impression-level, а за одно применение трансформера над пользователем, с teacher forcing

Target-aware action prediction

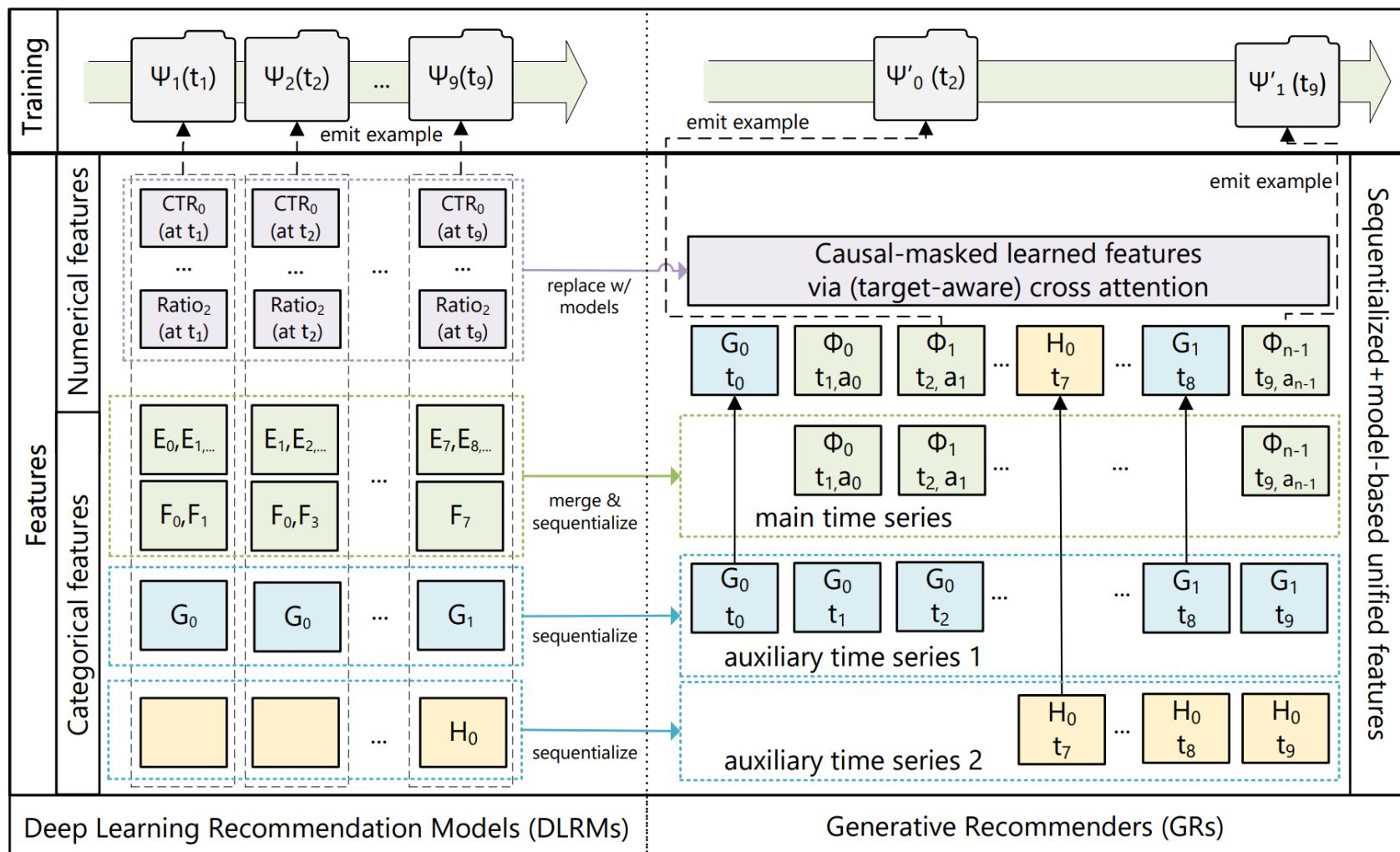


Практические детали

- Кроме показов и фидбека в истории есть признаки пользователя и контекст
 - Признаки пользователя вставляются в начало посл-ти
 - Когда появляется новый контекст или меняется старый – вставляется событие в историю
- Action – это не дискретный токен, а набор сигналов, предсказываемых одновременно
- Обучение хронологическое, по сессиям
 - То есть не за одно применение трансформера



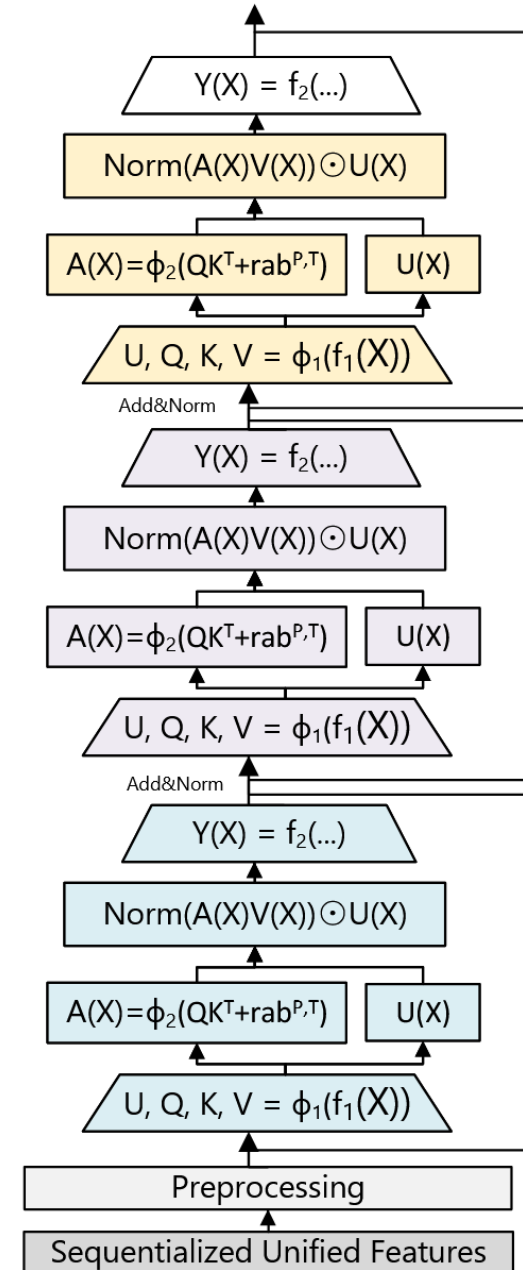
Практические детали



HSTU encoder

- Самые полезные признаки в ранжировании – user-item счетчики
- Трансформер плохо их моделирует из-за софтмакс нормализации
 - А еще плохо справляется с нестационарными распределениями
- Авторы предложили новый энкодер – HSTU
 - Убрали softmax и FFN-слои
 - Добавили в attention матрицу U для поэлементных умножений, похожих на feature interaction слой

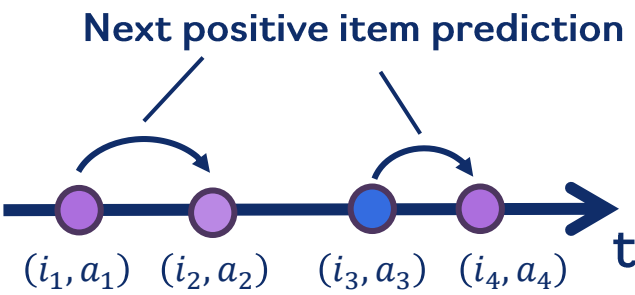
Избавились от feature engineering!



HSTU retrieval

Для генерации кандидатов авторы строят стандартную SASRec-подобную модель, но с HSTU энкодером

- В истории одно событие – пара (item, action)
- Предсказываем только следующие положительные взаимодействия
 $P(item \mid history, item \text{ is positive})$



Task		Specification (Inputs / Outputs)
Ranking	x_i s	$\Phi_0, a_0, \Phi_1, a_1, \dots, \Phi_{n_c-1}, a_{n_c-1}$
	y_i s	$a_0, \emptyset, a_1, \emptyset, \dots, a_{n_c-1}, \emptyset$
Retrieval	x_i s	$(\Phi_0, a_0), (\Phi_1, a_1), \dots, (\Phi_{n_c-1}, a_{n_c-1})$
	y_i s	$\Phi'_1, \Phi'_2, \dots, \Phi'_{n_c-1}, \emptyset$ $(\Phi'_i = \Phi_i \text{ if } a_i \text{ is positive, otherwise } \emptyset)$

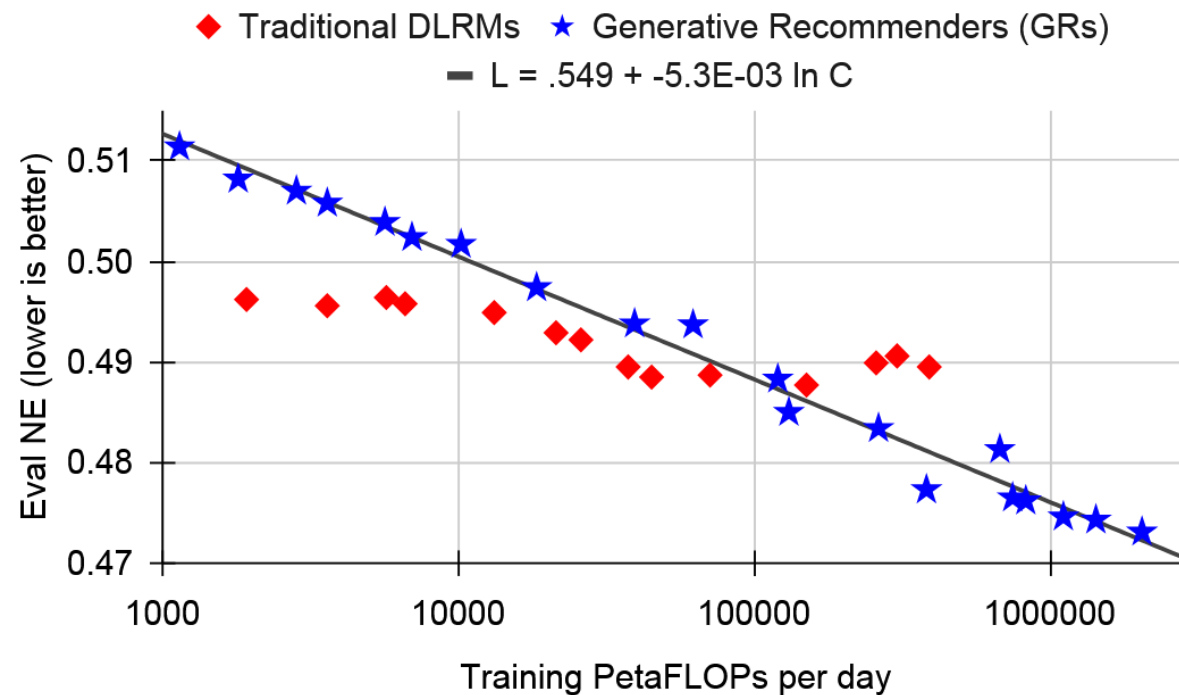
HSTU retrieval

Table 4. Evaluations of methods on public datasets in multi-pass, full-shuffle settings.

	Method	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2853	.5474	.7528	.1603	.2498
	HSTU	.3097 (+8.6%)	.5754 (+5.1%)	.7716 (+2.5%)	.1720 (+7.3%)	.2606 (+4.3%)
	HSTU-large	.3294 (+15.5%)	.5935 (+8.4%)	.7839 (+4.1%)	.1893 (+18.1%)	.2771 (+10.9%)
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	.3567 (+22.8%)	.6149 (+11.8%)	.8076 (+5.5%)	.2106 (+30.0%)	.2971 (+17.9%)
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	.0469 (+60.6%)	.1066 (+46.2%)	.1876 (+33.9%)	.0257 (+65.8%)	.0508 (+45.1%)

Масштабирование

- Показали scaling: прирост качества вплоть до 176 млн параметров в энкодере
 - До этой статьи все продвинувшие ранжирующие модели имели небольшие энкодеры (десятки миллионов параметров)
- Также увеличили анализируемую в модели длину истории до 8 тысяч событий



Результаты

Очень большой прирост целевых метрик на одном из главных продуктов

Methods	Offline NEs		Online metrics	
	E-Task	C-Task	E-Task	C-Task
DLRM	.4982	.7842	+0%	+0%
DLRM (DIN+DCN)	.5053	.7899	—	—
DLRM (abl. features)	.5053	.7925	—	—
GR (interactions only)	.4851	.7903	—	—
GR	.4845	.7645	+12.4%	+4.4%

Case study: ARGUS

Сейчас это основная парадигма, в которой учат трансформерные модели для рекомендаций в Яндексе:

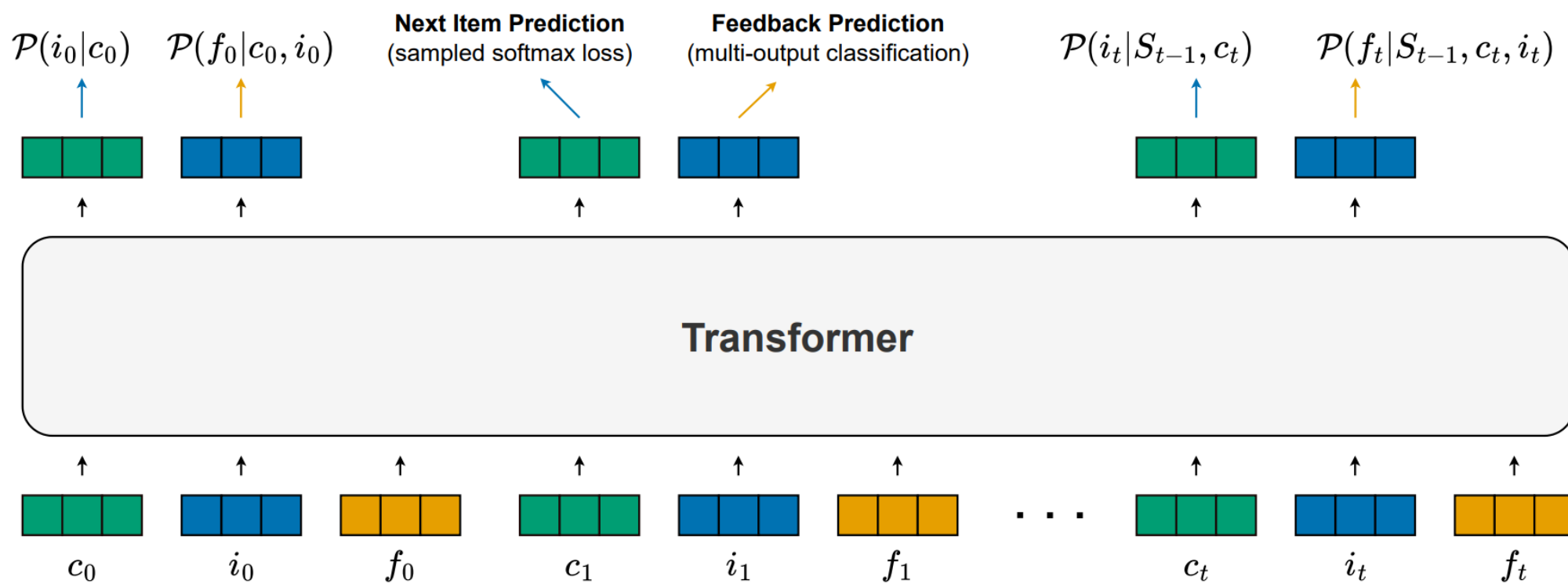
- ARGUS придумывала наша команда
- Я вдохновлялся HSTU-статьей и scaling hypothesis
- Основное изменение – перешли с impression-level обучения на авторегрессивные модели с teacher forcing
 - Это позволило сильно замасштабировать модели за те же ресурсы и показать scaling law

[\(arXiv\) Scaling Recommender Transformers to One Billion Parameters](#)

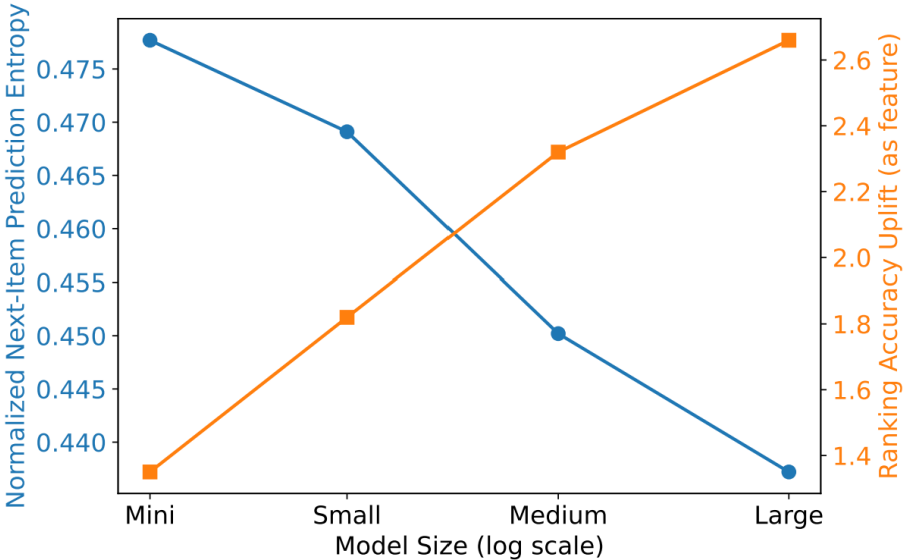
[\(habr\) ARGUS: как масштабировать рекомендательные трансформеры](#)

[\(DataFest\) Масштабирование рекомендательных систем](#)

ARGUS: архитектура



ARGUS: offline результаты



Encoder			Pre-train			Ranking Fine-tune (pair accuracy uplift)	
Model	Configuration	#Params	Feedback Prediction (normalized entropy)		Next-Item Prediction (normalized entropy)	Standalone	Feature
			Consumption	Engagement			
Mini	L4 H256	3.2M	0.5830 (-0.00%)	0.5855 (-0.00%)	0.4777 (-0.00%)	-7.49%	+1.35%
Small	L6 H512	18.9M	0.5756 (-1.28%)	0.5707 (-2.51%)	0.4691 (-3.81%)	-6.29%	+1.82%
Medium	L10 H1024	126.0M	0.5690 (-2.41%)	0.5556 (-5.10%)	0.4502 (-7.68%)	-5.33%	+2.32%
Large	L20 H2048	1.007B	0.5631 (-3.43%)	0.5436 (-7.15%)	0.4372 (-10.36%)	-4.78%	+2.66%
HSTU	L24 H1024	176.0M	0.5684 (-2.51%)	0.5553 (-5.15%)	0.4488 (-7.99%)	-5.39%	+2.34%

ARGUS: online результаты

Deployment	Context Length	Encoder		TLT	Like Likelihood
		Configuration	#Params		
Offline V1	512	L6 H512	18.9M	+0.52%	+1.11%
Offline V2	1024	L6 H512	18.9M	+1.00%	+0.73%
Offline V3	1024	L6 H512	18.9M	+0.73%	+5.00%
Real-time V1	1024	L4 H256	3.2M	+0.32%	+1.38%
Offline V4 (ARGUS)	8192	L10 H1024	126.0M	+2.26%	+6.37%

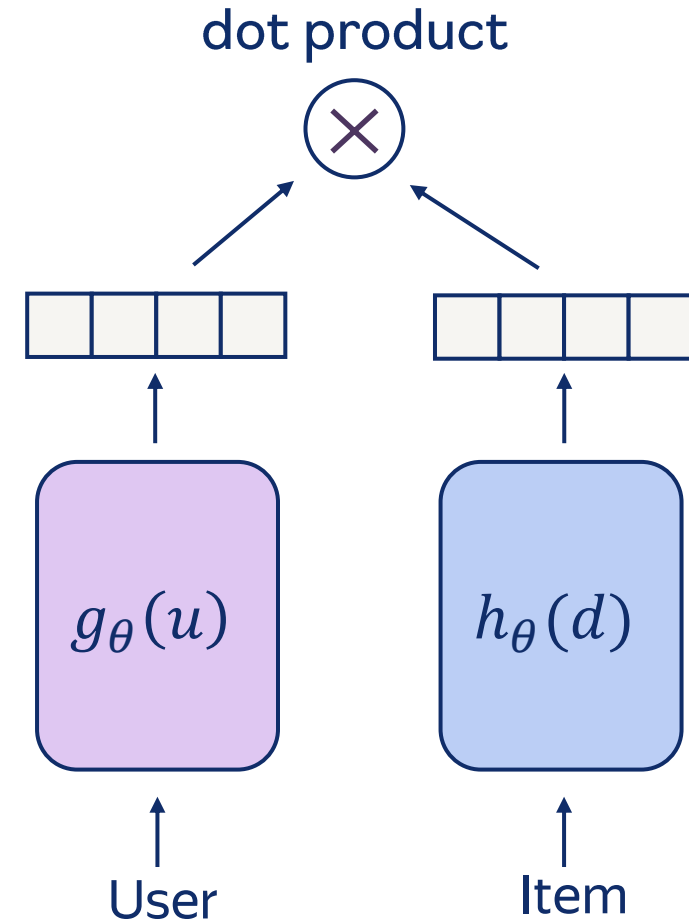
ARGUS дал самый большой прирост метрик от внедрения нейросетевой модели в Яндекс Музыке

Generative retrieval & semantic IDs

Часть 3

Нейросетевая генерация кандидатов

- Двухбашенные нейросети:
 - Раздельно кодируем пользователей и айтемы в эмбединги
 - Функция похожести – скалярное произведение
- Применение – быстрый поиск соседей по индексу
 - Нужно построить индекс по эмбедингам
 - И запускать в нём приближенную процедуру поиска соседей



Нейросетевая генерация кандидатов

- Проблема полного софтмакса – сложно учить модели из-за большого размера каталога
- Сложно внедрять:
 - Левая, “запросная” башня живёт в отдельном сервисе
 - Правую башню нужно пересчитывать по меняющемуся каталогу, перестраивать индекс и подгружать в сервис
 - Обновление модели должно происходить синхронно для левой и правой башни
- Выразительность двухбашенных моделей ограничена размерностью эмбедингов

DSI

- В поисковом домене столкнулись с теми же проблемами и придумали generative retrieval
- Решение: запомнить каталог в параметрах модели и обучить её генерировать элементы каталога в ответ на запросы
- Две задачи обучения:
 - Меморизация каталога: docid -> содержимое документа
 - Generative retrieval: запрос -> docid

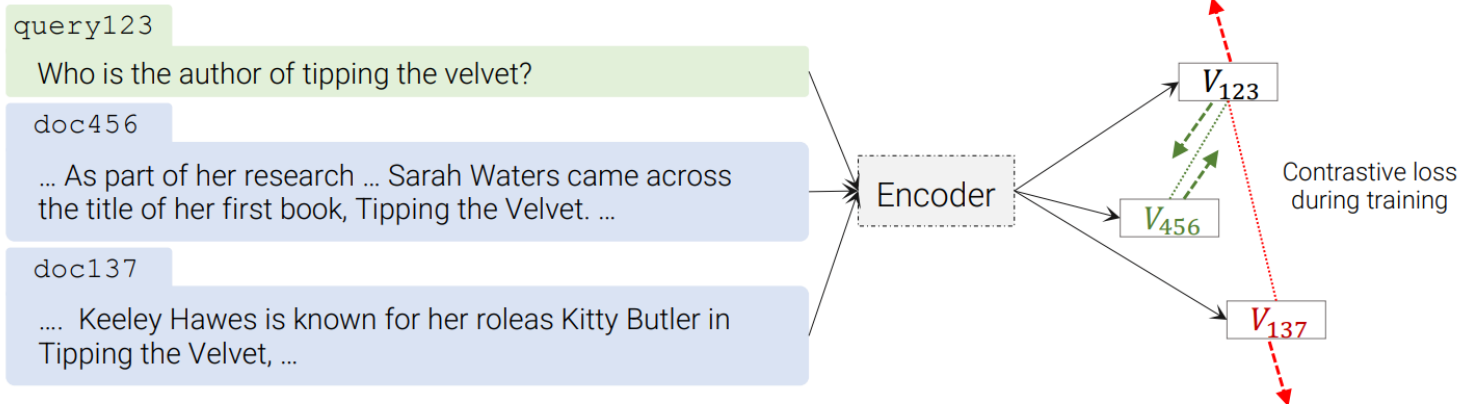
Opinion paper "Rethinking Search: Making Domain Experts out of Dilettantes"



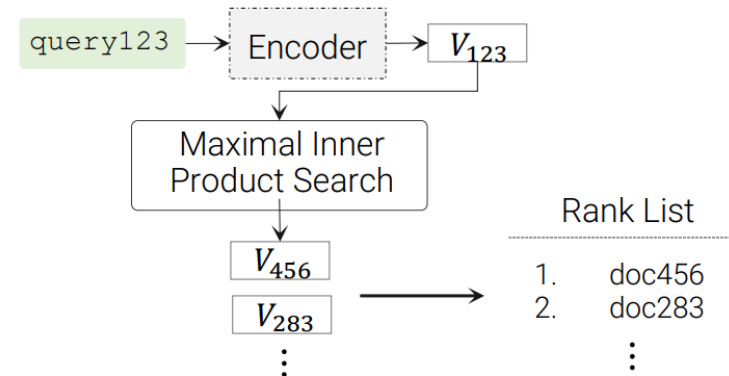
Как превратить большие языковые модели в полноценные информационные системы?

Encode-then-retrieve vs generative retrieval

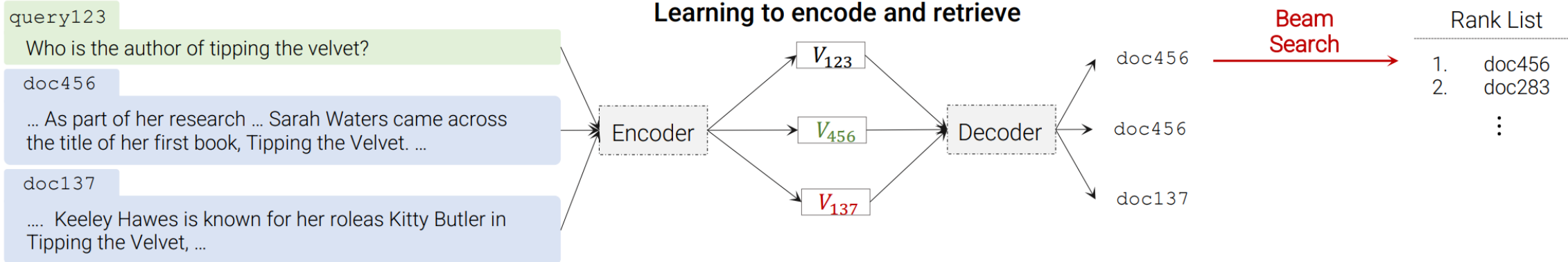
Learning to encode



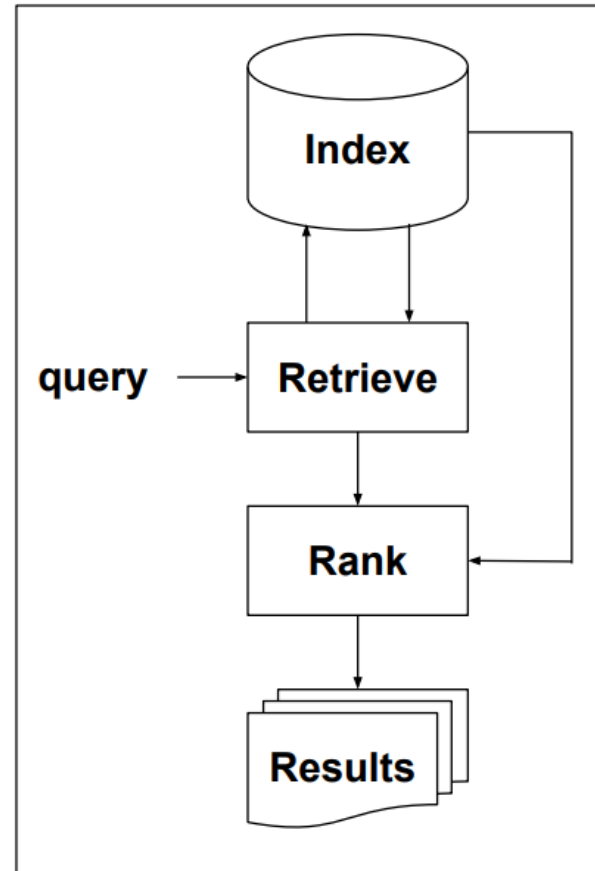
Retrieve



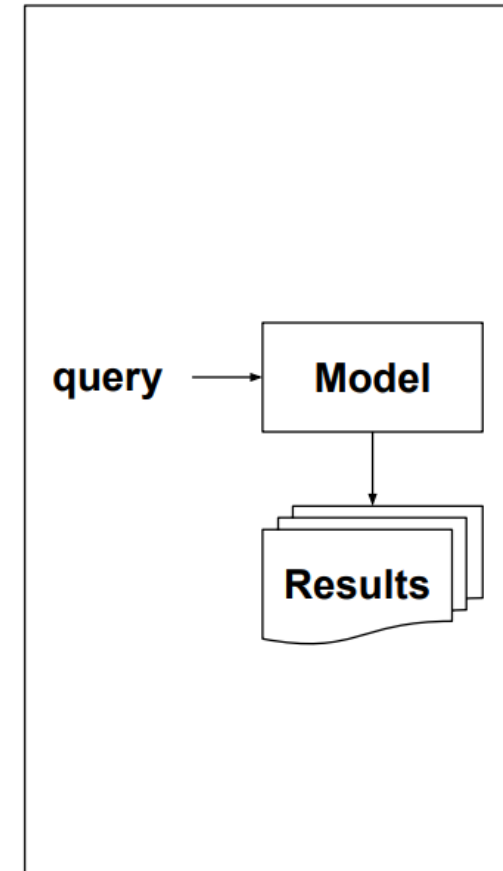
Learning to encode and retrieve



Retrieve-then-rank vs generative retrieval



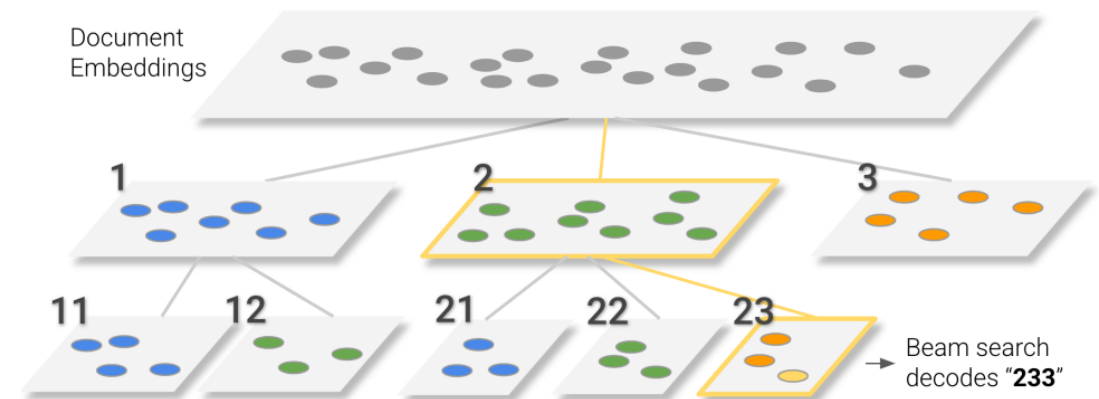
(a) Retrieve-then-rank



(b) Unified retrieve-and-rank

Generative retrieval

- Айтемы описываются последовательностью низкокардинальных идентификаторов $(s_0, s_1, s_2, s_3), s_i \in \text{Codebook}(i), |\text{Codebook}(i)| \ll \text{размер каталога}$
- Чтобы сформировать рекомендации, авторегрессивно генерируем эту последовательность
- Какие сложности будут со случайными идентификаторами?



Semantic ID

- Будем делать дискретные представления айтеров на основе эмбедингов
 - У похожих айтеров будут похожие semantic IDs
- Будем использовать контентные эмбединги
 - У нового контента сразу будут осмысленные semantic IDs
 - У айтеров с похожим контентом будут похожие semantic IDs
 - Рост качества на тяжелом хвосте
- Пусть у нас есть множество векторов айтеров. Как получить для них semantic IDs длины 1?

KMeans

- Алгоритм:
 1. Задаем количество кластеров, выбираем случайные объекты в качестве центроидов
 2. Ищем ближайший центроид для каждого вектора
 3. Пересчитываем центроиды исходя из получившихся кластеров
 4. Повторяем шаги 2 и 3 до сходимости
- Номер кластера, в который попал айтем – его semantic ID длины 1

Векторная квантизация

- Преобразование вектора в набор целых чисел называют **векторной квантизацией**

- Кодовая книга (кодбук) – набор векторов (кодовых слов)

$$C = \{c_1, \dots, c_n \mid c_i \in \mathbb{R}^D\}$$

- Целевой вектор описывается ближайшим кодовым словом

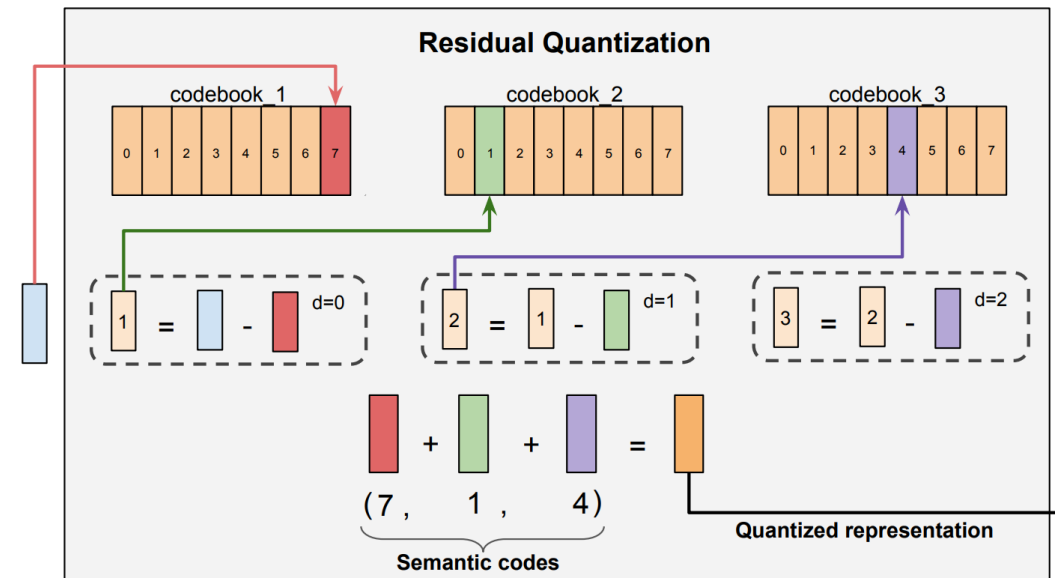
$$VQ(x) = \operatorname{argmin}_{c \in C} \|x - c\|$$

- С помощью KMeans можно построить кодовую книгу (центроиды – кодовые слова)
- Получение semantic IDs – это векторная квантизация

Как получить semantic IDs длины > 1 ?

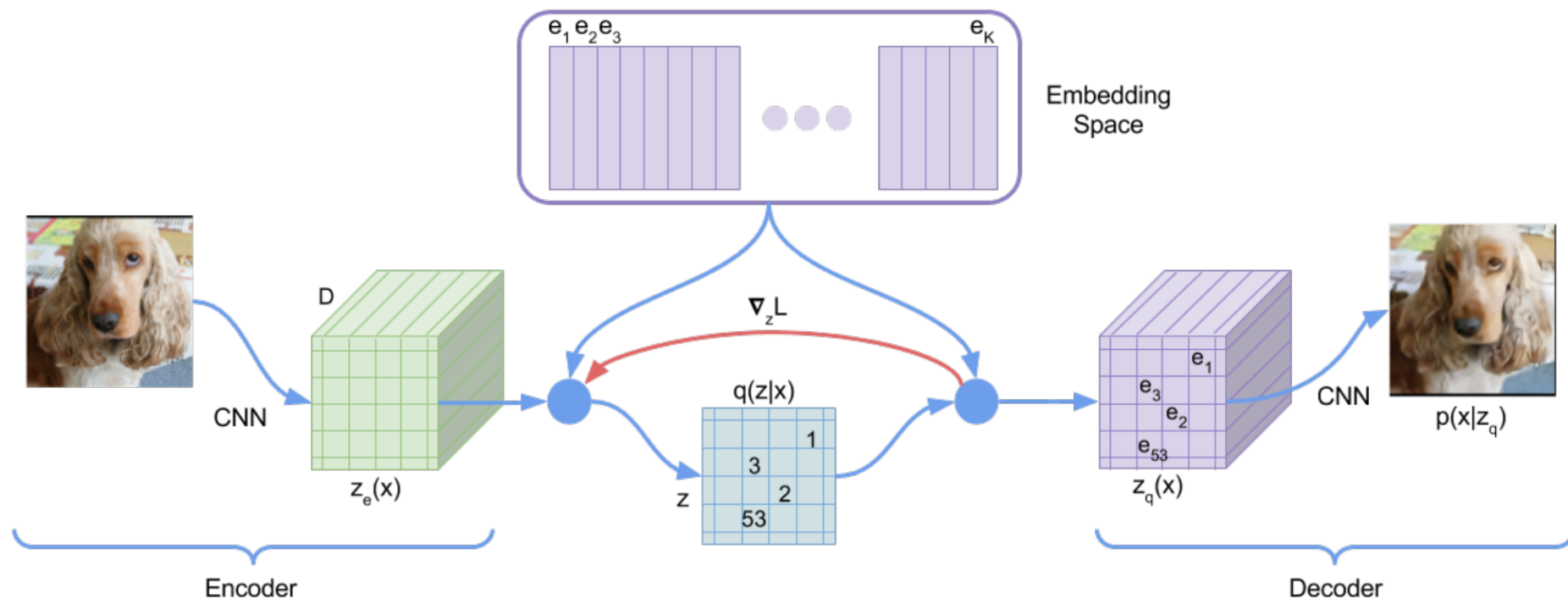
Residual KMeans

- Простой и эффективный метод получения semantic IDs
- Итеративно применяем KMeans:
 - В первый раз к исходным векторам
 - Затем вычитаем из векторов центроиды и применяем k-means к остаткам
 - Делаем итеративно до нужной длины semantic ID
- Иерархичность:
 - ранние токены кодируют более верхнеуровневую информацию
 - Каждый новый уровень кодирует остаток информации



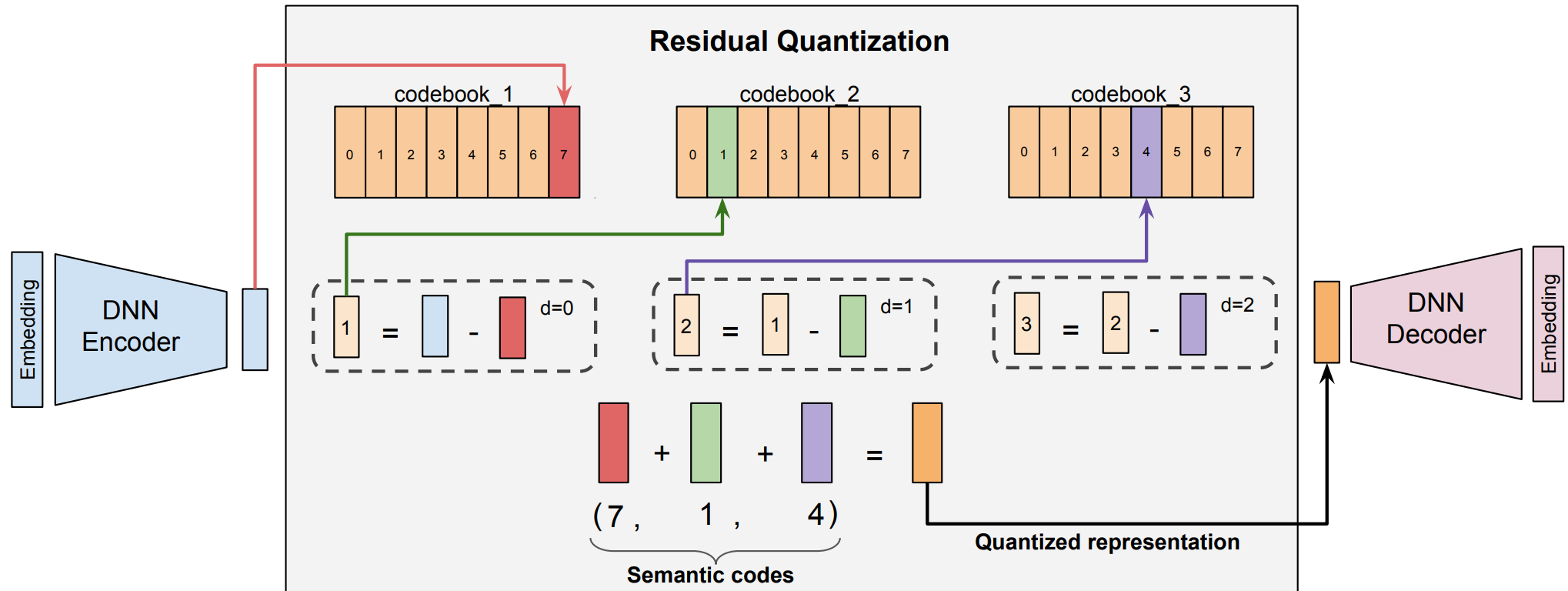
[Recommender Systems with Generative Retrieval](#)

VQ-VAE



$$\mathcal{L} = -\log p_{\theta}(x | z_q(x)) + \underbrace{\| \text{sg}[z_e(x)] - e_{k^*} \|^2}_{\text{codebook loss}} + \beta \underbrace{\| z_e(x) - \text{sg}[e * k^*] \|^2}_{\text{commitment loss}}$$

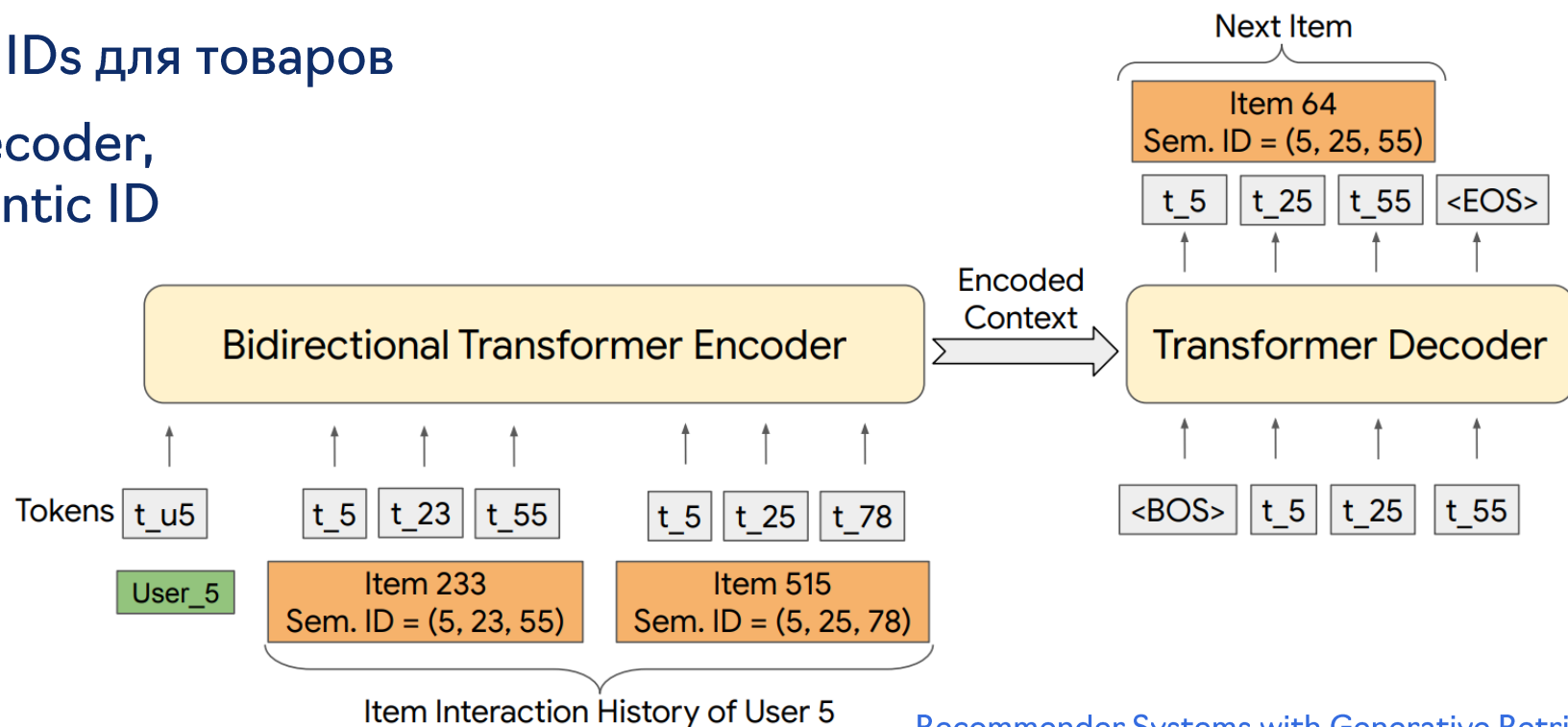
RQ-VAE



Case study: TIGER

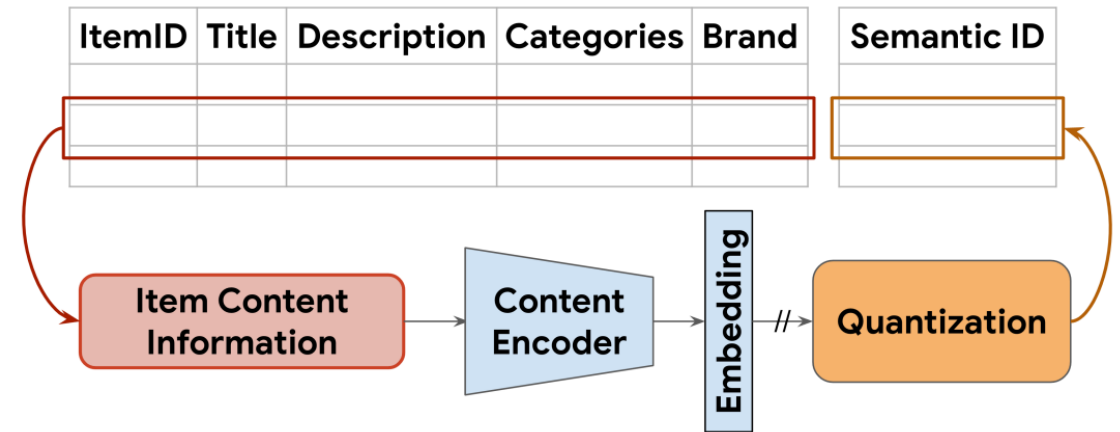
Первое применение generative retrieval для рекомендательных систем:

- Взяли Amazon reviews датасет
- Построили semantic IDs для товаров
- Обучили encoder-decoder, генерирующий semantic ID следующего айтема



TIGER: semantic IDs

- Собираем текстовые признаки айтема: title, price, brand, category
- Пропускаем через **Sentence-T5** → получаем эмбединг размерности 768
- Пропускаем через **RQ-VAE** → получаем кортеж кодов (c_0, c_1, c_2) — Semantic ID
- Для уникализации добавляем четвёртый “уникальный” токен → (c_0, c_1, c_2, m)



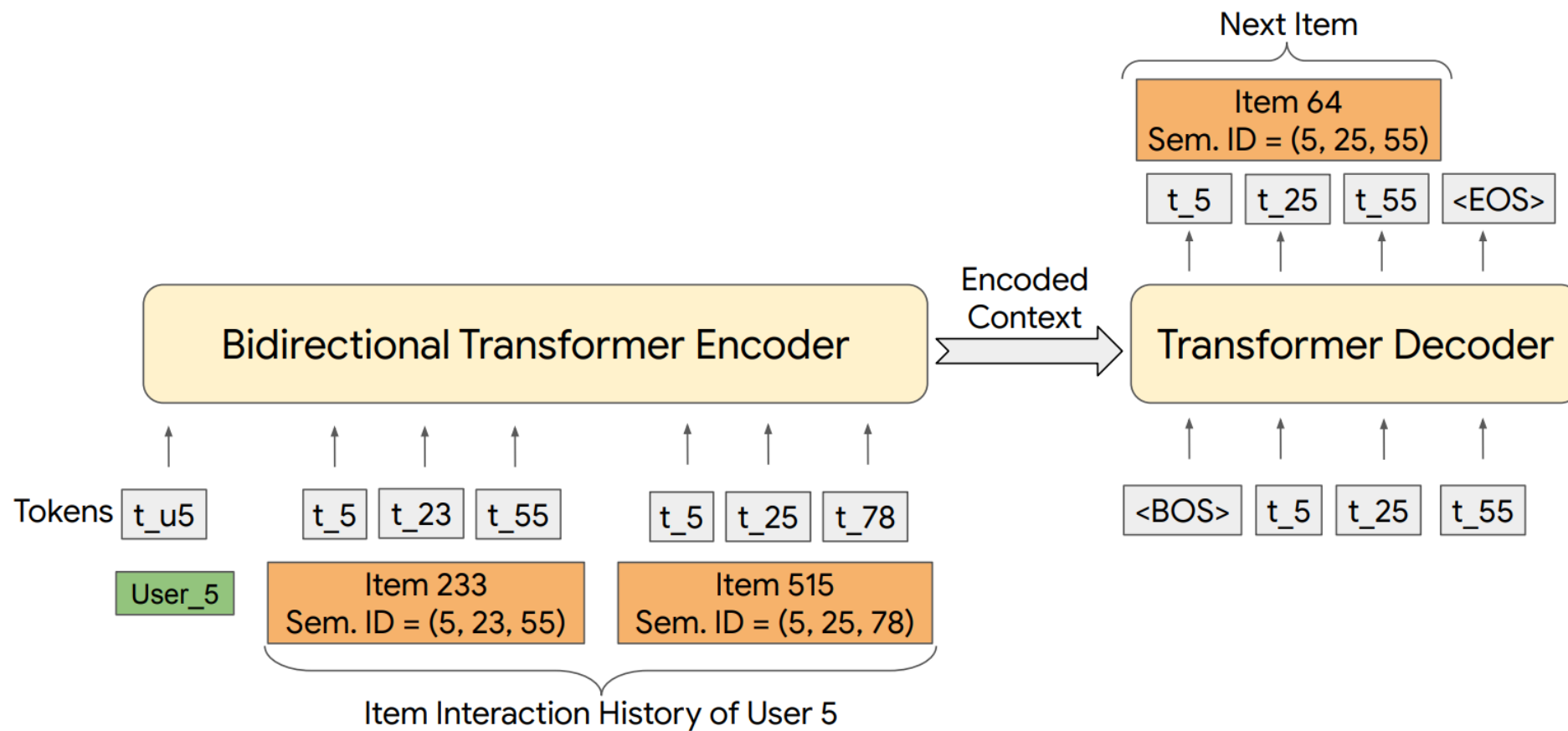
TIGER: архитектура

- История пользователя – последовательность семантических идентификаторов:
 $(c_0^1, \dots, c_3^1, c_0^2, \dots, c_3^2, \dots, c_0^n, \dots, c_3^n)$
- В начале user-токен (hashing trick на 2000 значений)
- Encoder-decoder transformer:
 - вход: последовательность Semantic ID (и user-токен)
 - выход: Semantic ID следующего айтема, токен за токеном

Применение

- авторегрессивная генерация
 $(c_0^{n+1}, \dots, c_3^{n+1})$
- **beam search** по префиксам Semantic ID
- используют **temperature-based sampling**, чтобы управлять diversity рекомендаций (выше температура → более разнообразные рекомендации)

TIGER: архитектура

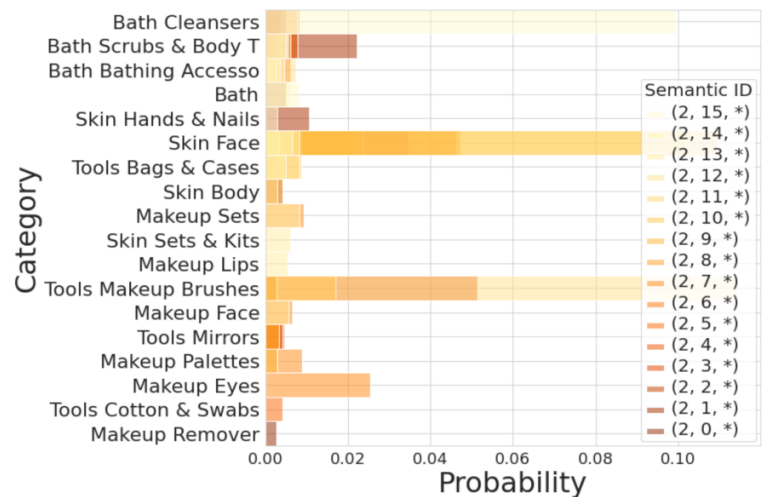
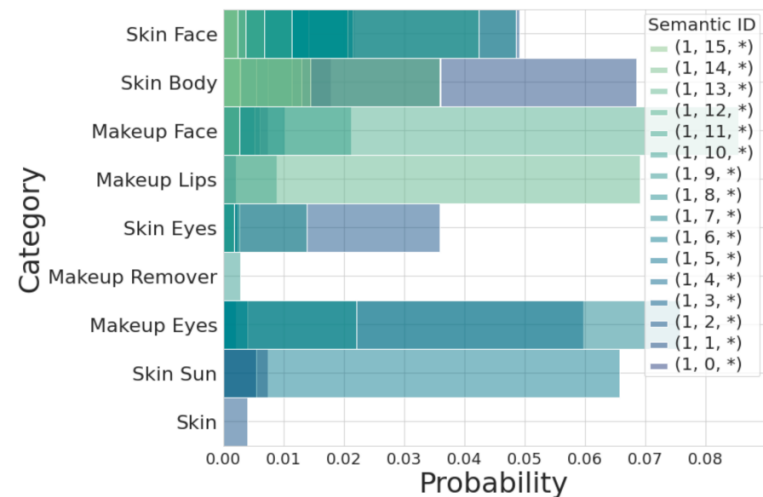
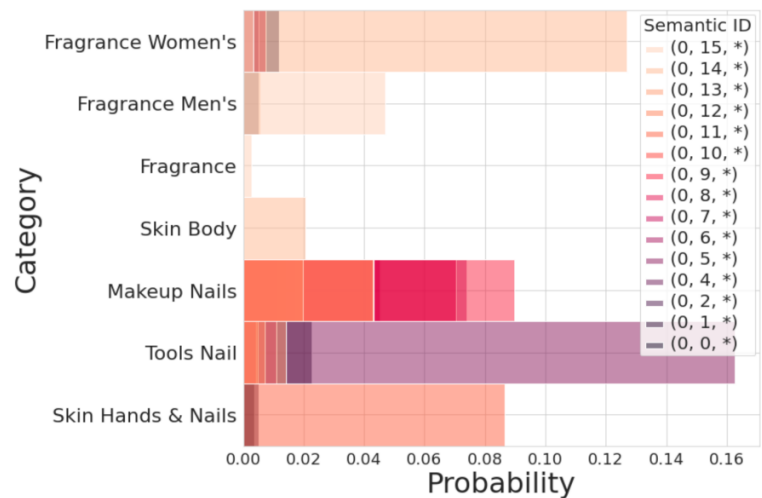
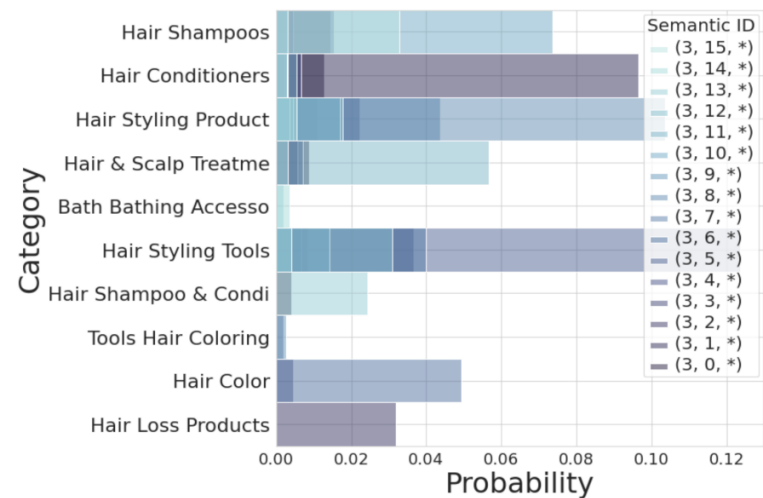


TIGER: результаты

Table 1: Performance comparison on sequential recommendation. The last row depicts % improvement with TIGER relative to the best baseline. Bold (underline) are used to denote the best (second-best) metric.

Methods	Sports and Outdoors				Beauty				Toys and Games			
	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10	Recall @5	NDCG @5	Recall @10	NDCG @10
P5 [8]	0.0061	0.0041	0.0095	0.0052	0.0163	0.0107	0.0254	0.0136	0.0070	0.0050	0.0121	0.0066
Caser [33]	0.0116	0.0072	0.0194	0.0097	0.0205	0.0131	0.0347	0.0176	0.0166	0.0107	0.0270	0.0141
HGN [25]	0.0189	0.0120	0.0313	0.0159	0.0325	0.0206	0.0512	0.0266	0.0321	0.0221	0.0497	0.0277
GRU4Rec [11]	0.0129	0.0086	0.0204	0.0110	0.0164	0.0099	0.0283	0.0137	0.0097	0.0059	0.0176	0.0084
BERT4Rec [32]	0.0115	0.0075	0.0191	0.0099	0.0203	0.0124	0.0347	0.0170	0.0116	0.0071	0.0203	0.0099
FDSA [42]	0.0182	0.0122	0.0288	0.0156	0.0267	0.0163	0.0407	0.0208	0.0228	0.0140	0.0381	0.0189
SASRec [17]	0.0233	0.0154	0.0350	0.0192	0.0387	<u>0.0249</u>	0.0605	0.0318	<u>0.0463</u>	<u>0.0306</u>	0.0675	0.0374
S ³ -Rec [44]	<u>0.0251</u>	<u>0.0161</u>	<u>0.0385</u>	<u>0.0204</u>	<u>0.0387</u>	<u>0.0244</u>	<u>0.0647</u>	<u>0.0327</u>	0.0443	0.0294	<u>0.0700</u>	<u>0.0376</u>
TIGER [Ours]	0.0264	0.0181	0.0400	0.0225	0.0454	0.0321	0.0648	0.0384	0.0521	0.0371	0.0712	0.0432
	+5.22%	+12.55%	+3.90%	+10.29%	+17.31%	+29.04%	+0.15%	+17.43%	+12.53%	+21.24%	+1.71%	+14.97%

TIGER: результаты



Проблемы и вызовы semantic ID

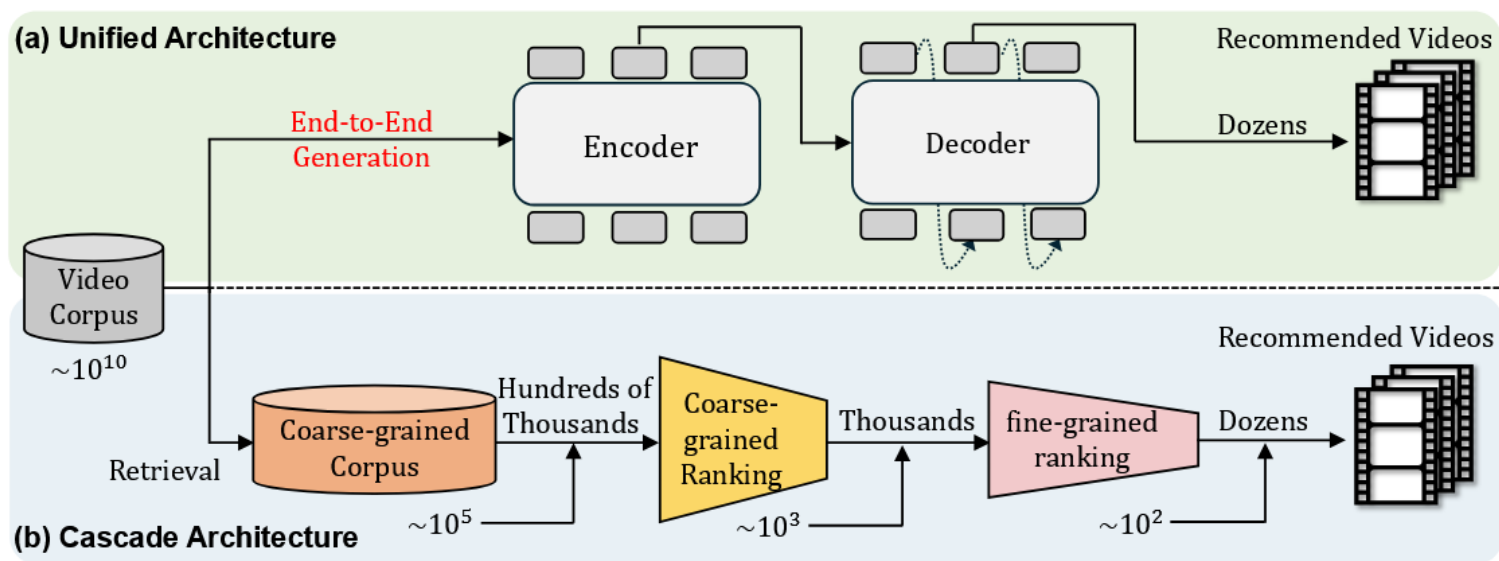
- Коллизии semantic ID
- Плохая утилизация кодбука (codebook collapse, hourglass phenomenon)
- Distribution drift
- Учёт коллаборативного сигнала
- End-to-end обучение с целевой задачей
- Beam search – медленный

Одностадийные рексистемы

Часть 4

Ресар многостадийности

- Стадия генерации кандидатов, преранжирование, ранжирование, переранжирование, блендинг, etc
- Это усложняет поддержку и развитие моделей, внедрение инноваций типа LLM



[OneRec: Unifying Retrieve and Rank with Generative Recommender and Iterative Preference Alignment](#)

OneRec

Инженеры из Kuaishou упростили рекомендательный стек до одной модели на основе generative retrieval:

- Строят semantic IDs поверх мультимодальных эмбеддингов айтеров с помощью R-KMeans
- Архитектура модели – энкодер-декодер
 - Энкодер кодирует историю пользователя
 - Декодер генерирует semantic IDs следующих пяти айтеров (session-wise подход)
- Предобучение: предсказание следующих пяти айтеров, но только на хороших сессиях
- В дообучении добавляют RL fine-tune (и задачу с предобучения тоже оставляют)

OneRec: обучение с подкреплением

RL fine-tuning:

- Обучают reward model по истории и сессии из пяти треков предсказывать время просмотра/лайки/etc, агрегируют в единую награду по всей сессии
- Сэмплируют из основной модели 128 траекторий, отбирают худшую и лучшую траекторию по мнению reward model
- DPO: увеличивают вероятность хорошей траектории относительно плохой

$$\begin{aligned}\mathcal{L}_{\text{DPO}} &= \mathcal{L}_{\text{DPO}}(\mathcal{S}_u^w, \mathcal{S}_u^l | \mathcal{H}_u) \\ &= -\log \sigma \left(\beta \log \frac{M_{t+1}(\mathcal{S}_u^w | \mathcal{H}_u)}{M_t(\mathcal{S}_u^w | \mathcal{H}_u)} - \beta \log \frac{M_{t+1}(\mathcal{S}_u^l | \mathcal{H}_u)}{M_t(\mathcal{S}_u^l | \mathcal{H}_u)} \right)\end{aligned}$$

OneRec: результаты

Внедрили на небольшой процент трафика как
одностадийную рексистему

- Длина истории во внедренной модели всего
256 событий

Model	Total Watch Time	Average View Duration
OneRec-0.1B	+0.57%	+4.26%
OneRec-1B	+1.21%	+5.01%
OneRec-1B+IPA	+1.68%	+6.56%

OneRec-v1: техрепорт

- Модель тоже называется OneRec; на первую статью не ссылаются
- Убрали долгосрочность из обучения
 - Теперь модель генерирует только один целевой айтем
 - Reward model – ранжирующая модель из многостадийного стека
 - Поменяли RL-алгоритм на GRPO
 - Фактически растят согласованность кандгена с ранжированием!
- Эффективная длина истории пользователя – 2 тысячи событий

OneRec-v1: результаты

- Внедрили на четверть трафика для short video recommendations
- Если добавить переранжирование reward моделью, метрики сильно растут

Scenarios	Online Metrics	OneRec	OneRec with RM Selection
Kuaishou	App Stay Time	+0.01%	+0.54%
	Watch Time	+0.07%	+1.98%
	Video View	+1.98%	+2.52%
	Like	-2.00%	+2.43%
	Follow	-2.88%	+3.24%
	Comment	-1.56%	+5.27%
	Collect	-0.61%	+2.93%
	Foward	+0.27%	+5.90%
Kuaishou Lite	App Stay Time	+0.06%	+1.24%
	Watch Time	+0.05%	+3.28%
	Video View	+2.40%	+3.39%
	Like	-2.64%	+1.49%
	Follow	-2.75%	+2.28%
	Comment	-2.23%	+3.20%
	Collect	-1.76%	+1.91%
	Foward	-1.86%	+3.48%

OneRec-v2

- Упростили архитектуру модели
 - Убрали трансформерный энкодер
 - Ускорили декодер: shared key-value embeddings, grouped query attention
- Добавили вторую RL задачу, на реальный фидбек:
 - Награда – нормализованное по пользователю время просмотра видеоролика
 - Это похоже на бандитов, тоже не долгосрочная награда
- Все еще используют модель как кандген, поверх есть reward модель

Scenarios	Online Metrics	OneRec-V2
Kuaishou	App Stay Time	+0.467%
	LT7	+0.069%
	Watch Time	+1.367%
	Video View	+0.331%
	Like	+3.924%
	Follow	+4.730%
	Comment	+5.394%
	Collect	+2.112%
	Forward	+3.183%
Kuaishou Lite	App Stay Time	+0.741%
	LT7	+0.034%
	Watch Time	+0.762%
	Video View	+0.259%
	Like	+5.393%
	Follow	+5.627%
	Comment	+5.013%
	Collect	+3.202%
	Forward	+7.958%

Выводы

- OneRec – case study хорошей рекомендательной модели на основе generative retrieval
- НО: она не одностадийная, и от двухстадийности скорее всего не получится отказаться
- Согласованность кандгена и ранжирования хорошо растит качество рекомендаций

Разговорные рексистемы

Часть 5

LLM x RecSys

- У LLM есть world knowledge, content understanding, reasoning
- LLM должны хорошо улучшать качество рекомендаций на тяжелом хвосте и помогать с холодным стартом
- Можно, например:
 - С помощью LLM делать content-based эмбединги айтемов
 - Размечать дополнительные данные для обучения
 - Делать переранжирование кандидатов с упором на разнообразие

Manipulation engines

- Даже текущие краткосрочные рекомендательные алгоритмы, которые заточены на мгновенный отклик, удерживают внимание пользователей
- Хорошо работающий RL = сильное влияние на будущее пользователей
- Метрика задаётся платформой и одинакова для всех

Fairness

- У каждого пользователя своя функция полезности
- Единственный способ повлиять на рекомендации для пользователя – оставлять фидбек, явный и неявный
 - Но неявный фидбек часто играет не на пользователя
- Как дать пользователю больше контроля над рекомендациями?
 - Добавлять настройки, фильтры, ползунки – работает плохо, ими не пользуются

Conversational recommender systems (CRS)

- Рексистема, с которой можно поговорить:
 - Пользователь явно формулирует свои пожелания
 - Может исправлять, уточнять, задавать ограничения
- В поиске уже есть похожая эволюция:
 - [8 blue links](#) → AI-generated answers
- Универсальные ассистенты (типа project Astra) должны уметь и общаться, и рекомендовать



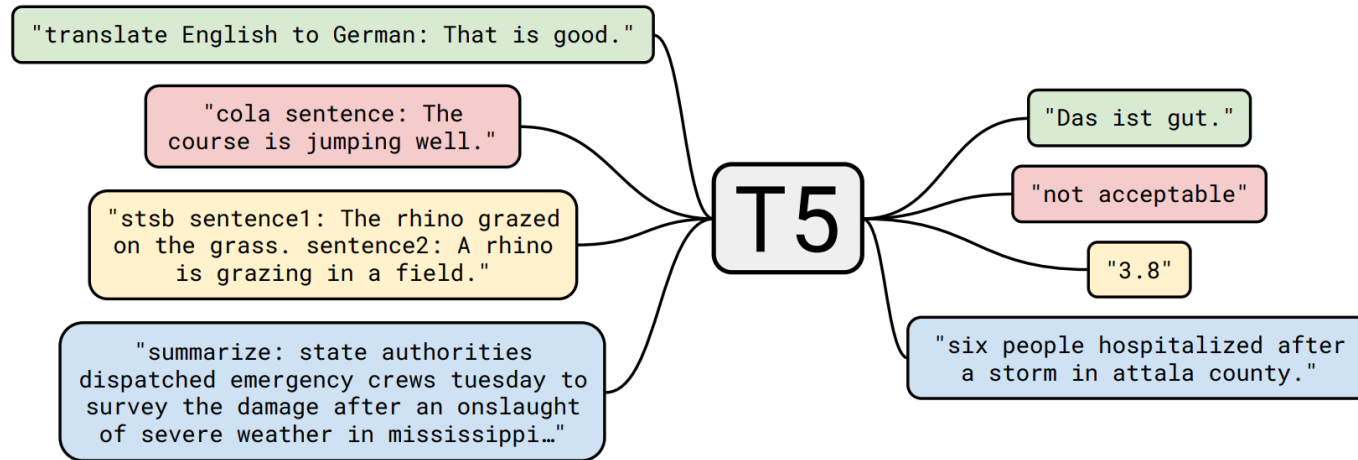
[Ed Chi - Google Gemini Era: Bringing AI to Universal Assistant and the Real World](#)

Каждый год во время своих выступлений хвалится, что у него на телефоне уже есть project Astra :)

CRS: первые шаги

- Можем попросить LLM что-то порекомендовать по запросу
- Персонализация: добавляем в prompt профиль пользователя на естественном языке
- Работает плохо, качество хуже чем у традиционных рекомендательных моделей
 - Матричная факторизация лучше чем LLM на MovieLens

T5: text-to-text transformer



- Всё в NLP формулируется как **text → text**: классификация, QA, суммаризация, перевод, и т.д.

Ключевые идеи:

- **Единый формат задач**: вход и выход — всегда строки текста
- **Encoder-decoder трансформер**, language modeling
- **Task prompts**: текстовый префикс говорит модели, какую задачу решать (например, “translate English to German: ...” vs “summarize: ...”)

[T5: Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer](#)

Case study: P5

Представим всё как текст:

- User-item взаимодействия, метаданные
- Решаем все задачи text-to-text с помощью encoder-decoder (T5)

Обучающий сэмпл

- **Input:** "инструкция + текстовые поля про юзера, айтема, историю, кандидатов..."
- **Target:** нужный ответ в текстовом виде (рейтинг, id айтема, объяснение, summary)

Instruction-based prompts - задаем задачу через текст:

- "Predict the star rating..."
- "Recommend the next item..."
- "Explain why the user likes..."

Примеры промптов

Rating prediction

"Given the user's history and the item, **predict the rating.**"

Sequential recommendation

"Given the interaction history of user U: $[i_1, i_2, \dots, i_n]$, **which item will the user interact with next?**"

Sequential с кандидатами

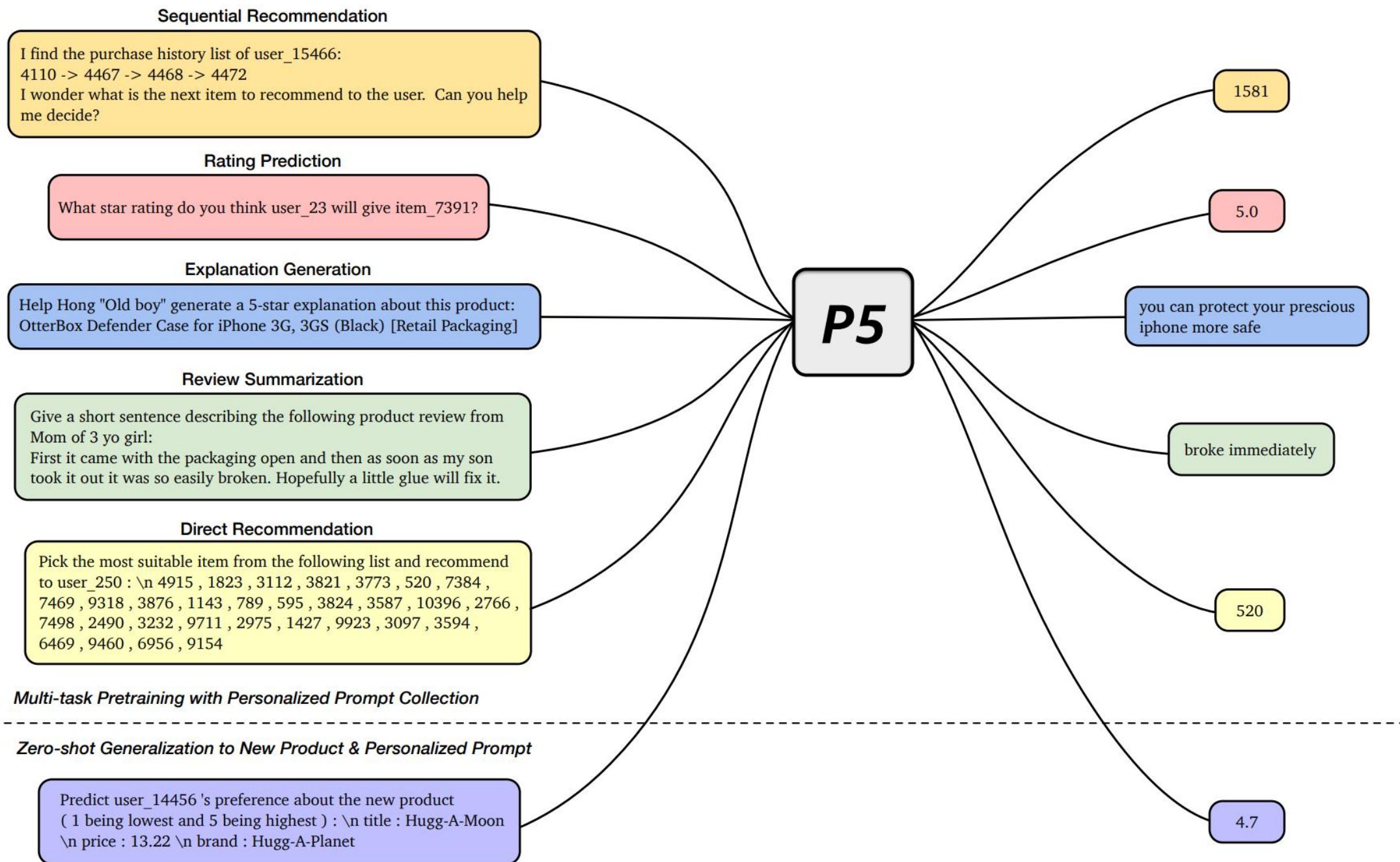
"Given the history ... and candidate items $[A, B, C, D]$, **which item will the user interact with next?**"

Binary next-item

"Given the history ..., **will the user interact with item X next? (yes/no)**"

Explanation / review / direct rec

- "Why does the user like item X?"
- "Summarize the reviews of item X."
- "Recommend an item for user U from $[A, B, C, D]$."

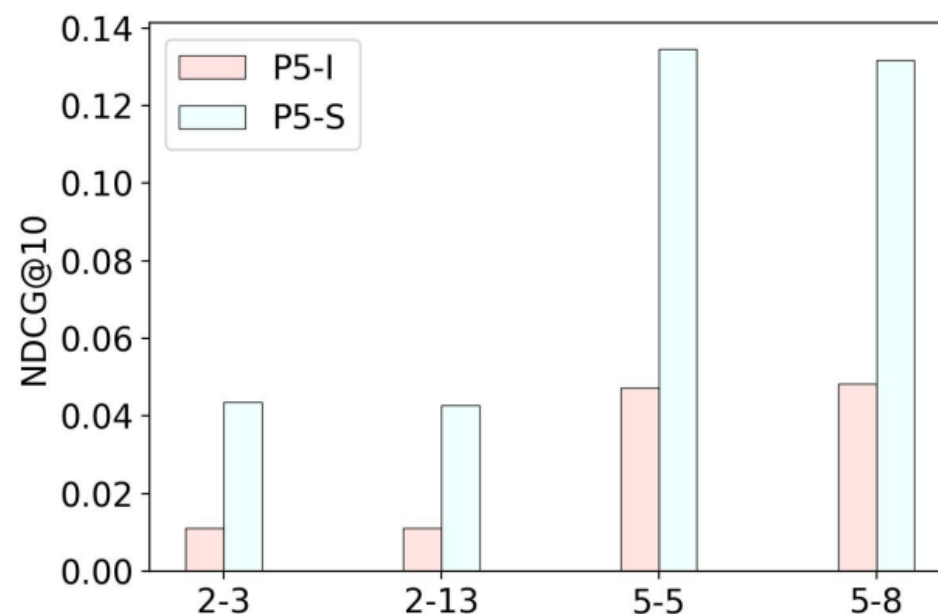


P5: представления пользователей и айтемов

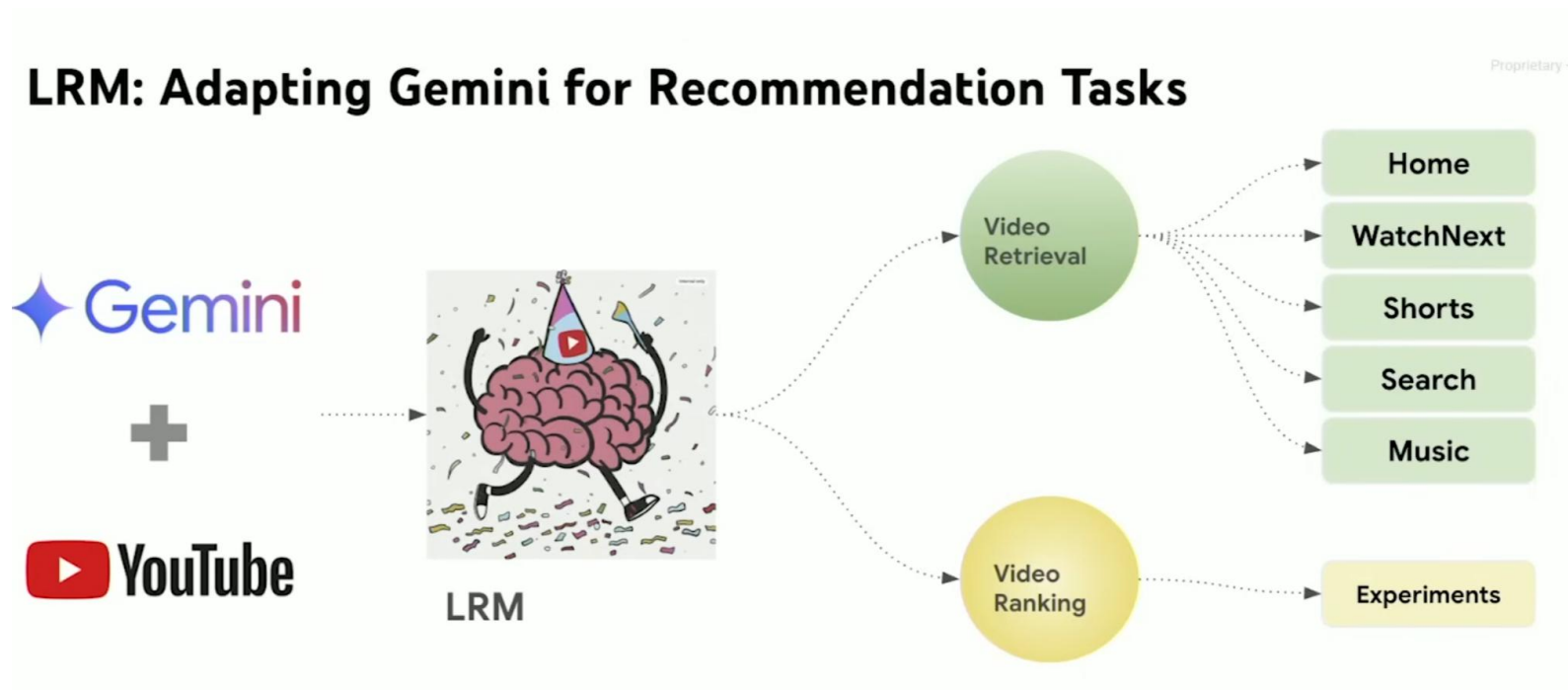
Пользователей и айтемы кодируем как строки "user_23", "item_7133":

- Токенизация: "user", "_", "23" / "item", "_", "7133"
- **Vocabulary gap**: пробовали кодировать каждого пользователя и айтем отдельным токеном, работает плохо

Нужны semantic IDs!



Разговорные рексистемы в YouTube



[Teaching Gemini to Speak YouTube: Adapting LLMs for Video Recommendations to 2B+DAU - Devansh Tandon](#)

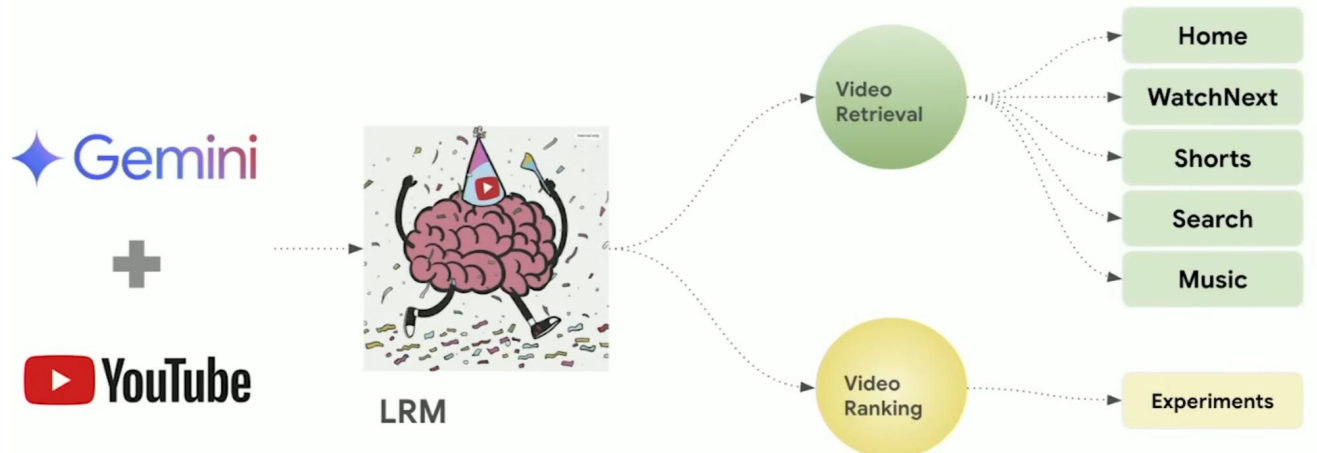
Case study: PLUM

Адаптация Gemini к рекомендациям YouTube:

1. Обучили RQ-VAE для эмбедингов YouTube видеороликов
2. Continuous pre-training чекпойнта Gemini на рекомендательных данных
3. Дообучение модели на задачу generative retrieval

Внедрили в качестве дополнительного источника кандидатов.

LRM: Adapting Gemini for Recommendation Tasks



[Teaching Gemini to Speak YouTube: Adapting LLMs for Video Recommendations to 2B+DAU - Devansh Tandon](#)

[PLUM: Adapting Pre-trained Language Models for Industrial-scale Generative Recommendations](#)

PLUM: CPT

- Половина датасета – поведенческие данные:
 - последовательности просмотров: [SID₁, SID₂, ...]
→ предсказываем следующий SID
 - модель учится рекомендовать
- Вторая половина – метаданные айтемов:
 - задачи вида “У видео <SID> следующее название: <title>”
или “Видео <SID> относится к такой-то категории”
 - модель учится понимать контент (индексация по аналогии с DS?)

Example user behavior training data

wh = <sid_1> <channel_name> <watch_ratio> <watch_time>
<hours_since_final_watch> <sid_2> <channel_name> ... || <sid_n>

SID + video title

Video <sid> has title (en): <video_title>

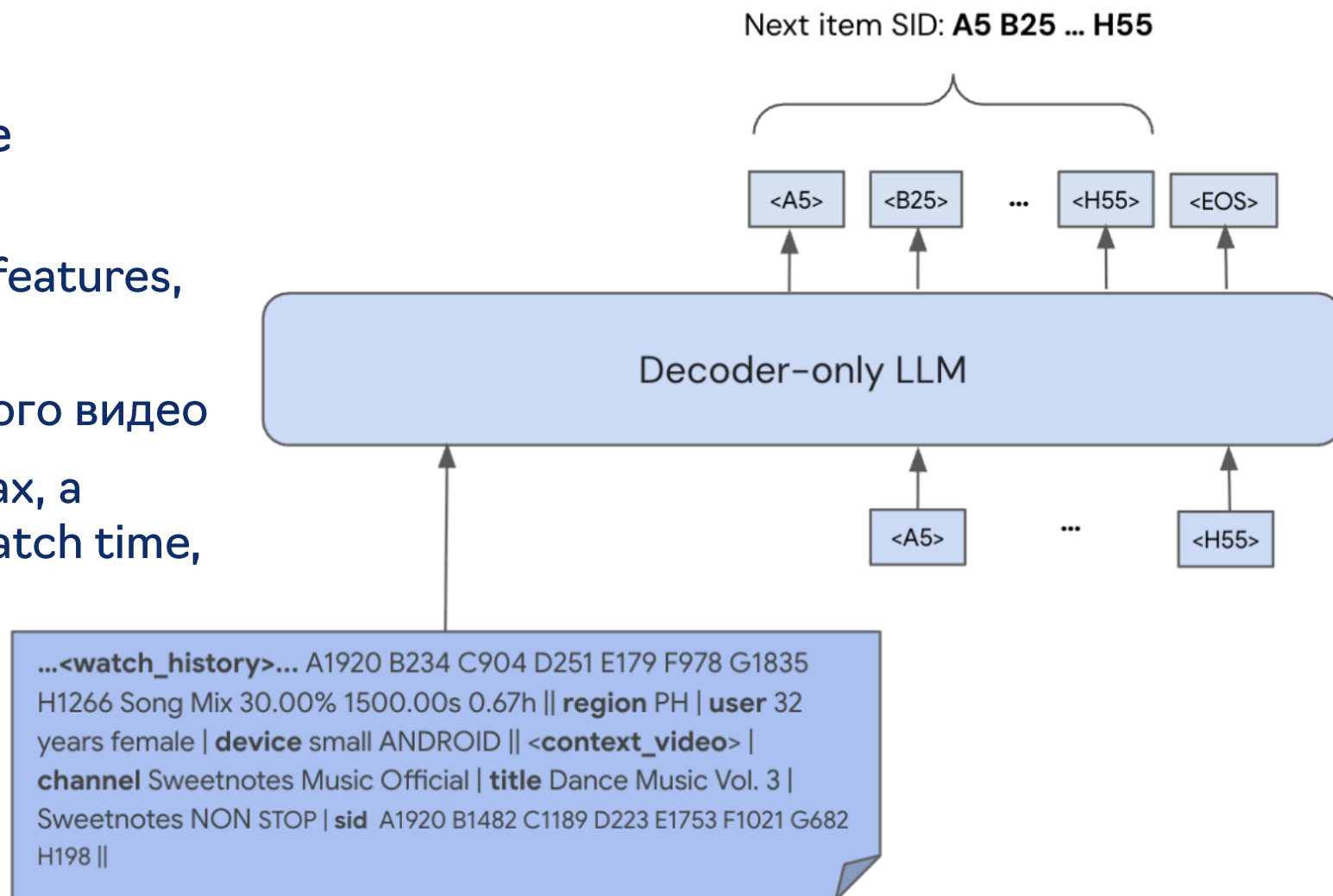
SID + video topics

The topics in video <sid> are: <topics>

PLUM: SFT

Дообучение на generative retrieval:

- вход: [watch history, user features, context features]
- выход: Semantic ID целевого видео
- обучают не на всех показах, а сэмплируют по reward (watch time, etc)



(a) Example 1



(1/3) B6t2IY9sUxA
Natural Remedies for Hypothyroidism
and Hashimoto's Disease
Dr. Josh Axe
556827 views



(2/3) BHEhXPuMmQI
Physicist Brian Cox explains quantum
physics in 22 minutes
Big Think
2365348 views



(3/3) iNyUmbmQQZg
Is it normal to talk to yourself?
TED-Ed
8682099 views

Video Thumbnail

Few-shot Input

The video A1991 ... H10 is about health and pain management.
The video A364 ... H37 is about the basics of quantum physics.
The video A37 ... H25 is about

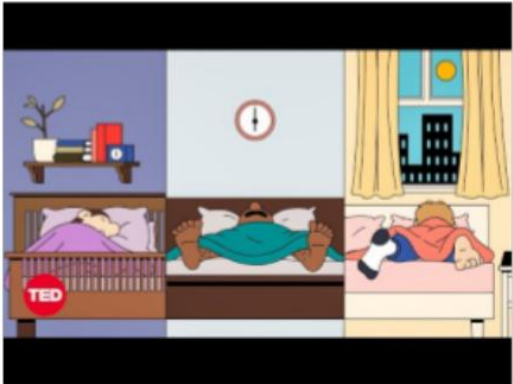
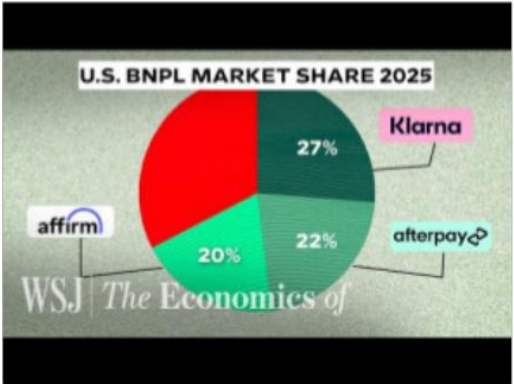
LLM-Initialized CPT Output

psychology and mind

Randomly-Initialized CPT Output

100% video A252 H7

(b) Example 2

 (1/3) fQUeDdaVoWo Do You Really Need 8 Hours of Sleep Every Night? Body Stuff with Dr. Jen Gunter TED TED 4129771 views	 (2/3) 3yC8WDjCIAU How 'Buy Now, Pay Later' Makes Billions From 'Free' Loans WSJ The Economics Of The Wall Street Journal The Wall Street Journal 904680 views	 (3/3) WSUj3PRvzzg Is being bilingual good for you brain? BBC Ideas BBC News 753351 views
---	--	---

Video Thumbnail

Few-shot Input

Video A37 ... H41 answers the question: do you really need 8 hours of sleep?
Video A1926 ... H8 answers the question: how buy-now pay-later loan business makes profit?
Video A1882 ... H32 answers the question:

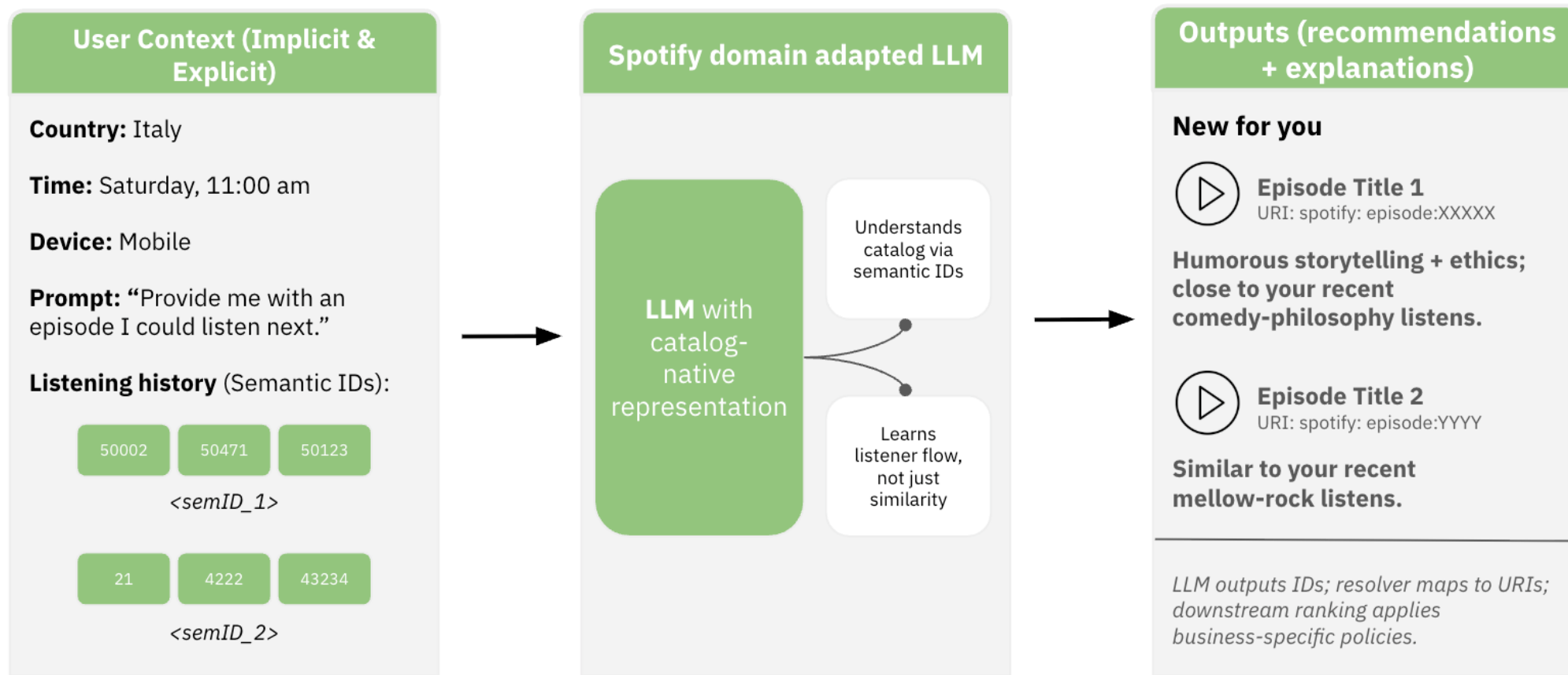
LLM-Initialized CPT Output

the impact of language in your life.

Randomly-Initialized CPT Output

- - - 100% -1999-11-16

Разговорные рекомендации в Spotify



[Teaching Large Language Models to Speak Spotify: How Semantic IDs Enable Personalization](#)

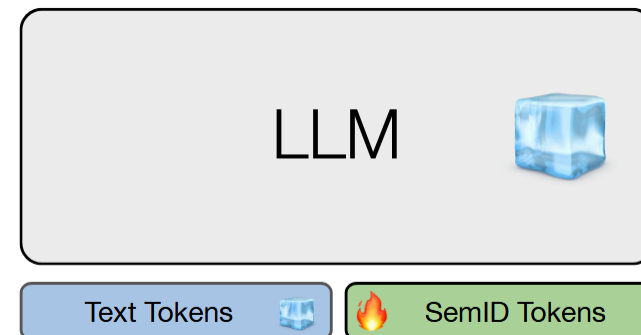
Case study: NEO

Модель от Spotify:

- Смотрят на рекомендации как на еще одну модальность в LLM
- Тоже есть semantic IDs для айтемов, причем для айтемов разного типа (артисты, подкасты, эпизоды, etc)
- Сначала происходит alignment стадия предобученной LLM:
 - Добавляют в словарь и выходной слой semantic IDs и дообучают на задачу меморизации каталога
- Затем дообучают как в P5 на семейство задач с промптами

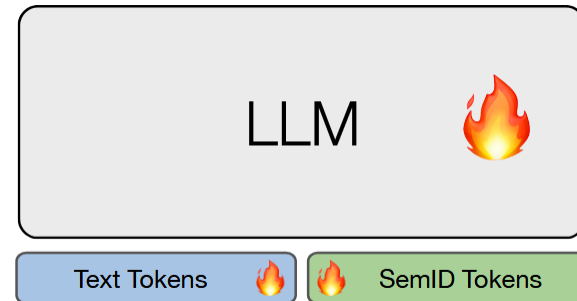
NEO: LLM & semantic IDs alignment

- Стадия меморизации из DSI
 - Добавляют semantic IDs в словарь LLM (и в выходной слой)
 - Замораживают все остальные параметры LLM
- Учат три задачи:
 - semantic ID -> text (verbalization), восстанавливают метаданные по semantic ID
 - text -> semantic ID (retrieval), по метаданным предсказывают semantic ID
 - semantic ID -> item type



NEO: instruction tuning

- Выделяют несколько задач и формируют для них промпты:
 - Next-item prediction
 - Text-based retrieval – примерно поисковый сценарий
 - Recommendation + recsplanation – вместе с рекомендацией генерируем объяснения
 - User understanding (interest profiling) – суммаризация пользовательских интересов
- Дообучают целиком всю LLM



NEO: результаты

Method	HR@K↑		NDCG@K↑	
	$k = 10$	$k = 30$	$k = 10$	$k = 30$
Episode Recommendation				
Baseline	-	-	-	-
NEO - mono	+57%	+41%	+80%	+60%
NEO - multi	+58%	+40%	+80%	+59%
Show Recommendations				
Baseline	-	-	-	-
NEO - mono	+20%	+2%	+46%	+30%
NEO - multi	+24%	+2%	+58%	+39%
Audiobook Recommendation				
Baseline	-	-	-	-
NEO - mono	+36%	+6%	+73%	+52%
NEO - multi	+46%	+14%	+97%	+66%
Text-based retrieval				
Baseline	0.45	0.51	0.37	0.38
NEO - mono	+45%	+36%	+62%	+58%
NEO - multi	+47%	+36%	+62%	+58%

Выводы

- Генеративные модели сэмпляют объекты из нужного нам распределения данных
- Генеративное ранжирование можно делать с помощью item-action interleaving
- Generative retrieval – следующий этап развития нейросетевой генерации кандидатов после двухбашенных моделей
- Приближаемся (или нет?) к одностадийным рексистемам
- Приближаемся к разговорным рексистемам
- Semantic IDs – важный элемент для generative retrieval, одностадийных рекомендательных систем, и для разговорных рексистем тоже

Давайте подытожим

Мы хотели:

1. Передать вам наши знания
2. Научить вас:
 1. Строить рекомендательные системы
 2. Обучать нейросети для рекомендаций
 3. Заниматься R&D и исследованиями

Давайте подытожим

В рамках курса, мы обсудили много всего:

1. Введение в рексистемы
2. Кандидатогенерация и метрики
3. Ранжирование и метрики
4. Хранение и обработка данных
5. Нейросетевая генерация кандидатов
6. Нейросетевое ранжирование
7. Дизайн высоконагруженных рекомендательных систем
8. Рекомендательные трансформеры на практике
9. RL в рекомендациях
10. Case Studies рекомендательных продуктов
11. Тренды в RecSys (генеративные модели)

Для вас старались



Даниил
Ткаченко



Кирилл
Хрыльченко



Роман
Нигматуллин



Артём
Матвеев



Владимир
Байкалов



Алексей
Красильников



Владислав
Тыцкий



Руслан
Кулиев



Валентина
Бронер

Спасибо, что слушали курс!



... и не забывайте про тяжелые хвосты :)